



Bachelorarbeit

VirtIO-Treiberintegration für D3OS

vorgelegt von

Tim Riedlinger

aus Düsseldorf

Abteilung Betriebssysteme
Prof. Dr. Michael Schöttner
Heinrich-Heine-Universität Düsseldorf

02. Februar 2026

Erstgutachter: Dr. Fabian Ruhland
Zweitgutachter: Prof. Dr. Martin Mauve
Betreuer: Dr. Fabian Ruhland

Inhaltsverzeichnis

1	Einleitung	1
1.1	Zielsetzung	2
1.2	Verwendete Hilfsmittel	2
1.3	Aufbau	3
1.4	Begriffe	4
2	Technische Grundlagen	5
2.1	Virtualisierung und paravirtualisierte Geräte	5
2.1.1	Host, Gast und Hypervisor	5
2.1.2	Gerätemodelle	5
2.2	VirtIO-Architektur	6
2.2.1	VirtIO-Geräte	7
2.2.2	Virtqueues	8
2.2.3	Benachrichtigungssteuerung	11
2.3	VirtIO-GPU	12
2.3.1	3D-Beschleunigung und VirGL	13
2.4	VirtIO-Sound	15
2.4.1	Architektur und Warteschlangen	15
3	Architektur und Einbindung der VirtIO-Treiber	18
3.1	Integrationskonzept und Anforderungen	18
3.2	DMA-Speicherbereitstellung	19
3.3	HAL-Anbindung	20
3.4	Geräteinitialisierung	21
3.4.1	VirtIO-Erkennung	22
3.4.2	BAR-Mapping	23
3.4.3	Aktivierung und Interrupt-Anbindung	25
3.4.4	Treiberinstanzen und zentrale Bereitstellung	25
3.5	Interrupt- und Event-Anbindung	26
3.6	Globales Zugriffskonzept	28
3.7	VirtIO-Sound und Beispielanwendung	29
4	VirtIO-GPU und Erweiterungen	32
4.1	Erweiterung um VirGL und 3D-Kommandos	32
4.2	VirGL Smoke Test	35
4.3	Resize-Unterstützung	38
4.4	Demos: Kompatibilität und Synchronisation	39
5	Evaluation	42

5.1	Ausgabe des VirGL Smoke Tests	42
5.2	VirtIO-Sound (PCM-Playback)	45
5.3	Leistung: Rectangle-Demo	46
5.3.1	Messmethodik und Korrektur der Zeitbasis	46
5.3.2	Ergebnisse	47
5.4	Diskussion und Ausblick	48
6	Fazit	49
	Literatur	50
	Abbildungsverzeichnis	52
	Tabellenverzeichnis	53

Kapitel 1

Einleitung

Die Entwicklung von Betriebssystemen findet heute häufig in virtualisierten Umgebungen statt. Virtuelle Maschinen ermöglichen reproduzierbare Testaufbauten, vereinfachen Debugging und erlauben es, neue Kernel-Funktionen unabhängig von konkreter physischer Hardware zu evaluieren. Gerade bei Forschungs- und Lehrbetriebssystemen ist diese Reproduzierbarkeit wichtig, weil sie Experimente, automatisierte Tests und die Zusammenarbeit im Team erleichtert.

In diesem Umfeld hat sich **VirtIO** als zentraler Baustein etabliert. VirtIO beschreibt eine Familie paravirtualisierter Geräte, die speziell dafür entworfen wurden, in virtuellen Maschinen effizient bereitgestellt und genutzt zu werden. Statt komplexe “klassische” Hardware (z. B. Netzwerkkarten oder Grafikkartenmodelle) vollständig zu emulieren, stellt der Hypervisor standardisierte VirtIO-Geräte zur Verfügung, die vom Gast über einen klar definierten Mechanismus angesprochen werden. Die Spezifikation ist als offener Standard bei OASIS organisiert.

Im Kontext dieser Arbeit steht **D3OS** (Distributed Data-Driven OS), ein am Lehrstuhl entwickeltes, Rust-basiertes verteiltes Betriebssystem, das auf skalierbares Big-Data-Computing auf moderner Rechenzentrums-Hardware abzielt. D3OS verfolgt dabei den Anspruch, Betriebssystemdesign stärker an die Anforderungen datengetriebener Workloads und aktueller Hardwarelandschaften anzupassen [1].

Für D3OS löst VirtIO ein praktisches Kernproblem: Ohne eine solche Standard-Schnittstelle müsste ein Lehr- und Forschungsbetriebssystem entweder viele hardware-spezifische Treiber selbst entwickeln und warten, oder auf weniger geeignete/ineffizientere Emulationsmodelle ausweichen. Beides erhöht die Komplexität stark und bindet Entwicklungszeit an Peripherie-Themen, statt an den eigentlichen OS-Fragestellungen. VirtIO fungiert hierbei als standardisierte Schnittstelle, die es D3OS ermöglicht, hardwareunabhängig mit einem einheitlichen Treibermodell in virtualisierten Umgebungen zu agieren. In der Folge werden die Testbarkeit und Portabilität des Systems optimiert, was die Effizienz in der akademischen Lehre und Weiterentwicklung nachhaltig steigert.

Um diesen Nutzen in D3OS verfügbar zu machen, bietet sich die Nutzung vorhandener, `no_std`-fähiger VirtIO-Treiberbausteine in Rust an, die bereits mehrere gängige VirtIO-Geräteklassen abdecken.

1.1 Zielsetzung

Ziel dieser Bachelorarbeit ist die Integration einer VirtIO-Treiberbasis in D3OS. Hierzu wird die Rust-Crate **virtio-drivers (rcore-os)** [2] angebunden, sodass grundsätzlich eine Vielzahl von VirtIO-Geräten über ein einheitliches Treibermodell angesprochen werden kann. Dadurch entsteht eine wiederverwendbare Grundlage, um VirtIO-Geräte künftig mit geringem Zusatzaufwand zu aktivieren und in studentischen Projekten weiter auszubauen.

Auf dieser Basis fokussiert die Arbeit zwei konkrete Funktionsbereiche:

- Grafikunterstützung über VirtIO-GPU: Der in der Crate vorhandene GPU-Treiber wird nutzbar gemacht und anschließend funktional erweitert, um VirGL-Fähigkeiten zu unterstützen und zu demonstrieren.
- Audioausgabe über VirtIO-Sound: Zusätzlich wird VirtIO-Sound verwendet, um eine Audioausgabe bereitzustellen. Darauf aufbauend wird exemplarisch ein einfacher Audioplayer realisiert, der die praktische Nutzbarkeit der Audio-Schnittstelle demonstriert.

Zur Einordnung und Bewertung der grafischen Integration werden die bestehenden Demoprogramme (Pong- und Rectangle-Demo) aus der Arbeit von Nikita Elrich so angepasst, dass sie auch mit der Crate-basierten Implementierung lauffähig sind. Außerdem werden Treiber- und Demo-Code erweitert, um Resize (dynamische Änderungen der Auflösung) zu unterstützen. Abschließend erfolgt ein Performance-Vergleich zwischen einer WSL2-Umgebung und einem nativen Ubuntu-Host, wobei identische Testszenarien und Messbedingungen verwendet werden, um die Auswirkungen der unterschiedlichen Hypervisor-Architekturen auf die grafische Ausgabe zu evaluieren.

Für die VirGL-Unterstützung wird darüber hinaus ein minimalistischer Testaufbau umgesetzt, der einen einfachen VirGL-Command-Stream generiert und an den Host sendet. Damit wird gezeigt, dass der Host die Kommandos annimmt und verarbeitet, was die grundsätzliche Umsetzbarkeit des VirGL-Ansatzes belegt.

Ein vollständiger Funktionstest aller in virtio-drivers enthaltenen Treiber ist nicht Gegenstand dieser Arbeit, jedoch ist die Integration so ausgelegt, dass zusätzliche VirtIO-Geräte einheitlich initialisiert, und anschließend mit geringem Anpassungsaufwand in eigenen Anwendungen oder studentischen Erweiterungen genutzt werden können.

1.2 Verwendete Hilfsmittel

Im Rahmen dieser Arbeit wurden verschiedene digitale Hilfsmittel eingesetzt. Zur Unterstützung bei der Entwicklung und Überarbeitung von Code sowie bei technischen Fragestellungen kamen ChatGPT (5.2) und Google Gemini 3 zum Einsatz. Darüber hinaus wurden diese Werkzeuge punktuell zur Unterstützung bei der LaTeX-Formatierung und bei der sprachlichen Ausarbeitung genutzt, etwa zur Verbesserung von Formulierungen, Struktur und Verständlichkeit.

1.3 Aufbau

Kapitel 2 stellt die technischen Grundlagen bereit, die für das Verständnis der Arbeit erforderlich sind. Dazu werden Virtualisierung und paravirtualisierte Geräte eingeordnet, die VirtIO-Architektur mit Virtqueues und Benachrichtigungsmechanismen erläutert sowie die gerätespezifischen Konzepte von VirtIO-GPU (inkl. VirGL/3D-Erweiterung) und VirtIO-Sound vorgestellt.

Kapitel 3 beschreibt das Integrationskonzept der Treiberbasis virtio-drivers in D3OS und die dafür notwendigen Anpassungen im System. Im Fokus stehen dabei die Einbindung der Crate in die Kernelarchitektur, die Bereitstellung von DMA-fähigem Speicher, die Implementierung der HAL-Schnittstelle sowie die Geräteinitialisierung über PCI inklusive Mapping und Interrupt-Anbindung. Darauf aufbauend werden das Event-/Interrupt-Konzept, die zentrale Bereitstellung der Treiberinstanzen und die Umsetzung von VirtIO-Sound samt Beispielanwendung behandelt.

Kapitel 4 vertieft anschließend den GPU-Teil und dokumentiert die Erweiterungen am VirtIO-GPU-Treiber. Hierzu zählen die Ergänzungen für VirGL und 3D-Kommandos, der zugehörige Smoke Test als funktionsbezogener End-to-End-Nachweis, die Umsetzung von Resize-Unterstützung sowie Anpassungen an den Demos (Kompatibilität/Synchronisation) für die Nutzung der neuen Treiberbasis.

Kapitel 5 evaluiert die praktische Nutzbarkeit der Integration in drei Dimensionen: die Ausgabe des VirGL Smoke Tests, die Validierung der Audioausgabe über VirtIO-Sound (PCM-Playback) und eine quantitative Leistungsbewertung der Grafikpipeline anhand der Rectangle-Demo; die Ergebnisse werden abschließend diskutiert.

Kapitel 6 fasst die wesentlichen Ergebnisse der Arbeit zusammen und ordnet sie im Hinblick auf die eingangs formulierte Zielsetzung ein. Dabei wird bewertet, inwieweit die Integration der VirtIO-Treiberbasis in D3OS sowie die exemplarische Umsetzung von VirtIO-GPU (inkl. VirGL) und VirtIO-Sound erfolgreich realisiert werden konnten. Abschließend werden zentrale Einschränkungen der Arbeit benannt und ein Ausblick auf mögliche weiterführende Entwicklungen und Erweiterungen gegeben.

1.4 Begriffe

Zum besseren Verständnis der technischen Ausführungen werden im Folgenden zentrale Fachbegriffe und Abkürzungen kurz erläutert, die im Kontext der VirtIO-Treiberentwicklung und Systemprogrammierung verwendet werden.

- **Mutex (Mutual Exclusion):** Ein Synchronisationsobjekt, das sicherstellt, dass zu jedem Zeitpunkt nur ein Thread oder Prozess auf eine gemeinsame Ressource zugreifen kann.
- **Interrupt Storm:** Ein Zustand, bei dem das System so viele Unterbrechungsanforderungen (Interrupts) in kurzer Zeit erhält, dass es fast ausschließlich mit deren Verarbeitung beschäftigt ist und die normale Programmausführung stagniert.
- **DMA (Direct Memory Access):** Ein Verfahren, bei dem Hardware-Komponenten direkt auf den Arbeitsspeicher zugreifen können, ohne die CPU für den Datentransfer zu belasten.
- **Scatter-Gather:** Eine Technik beim DMA-Transfer, die es ermöglicht, Daten aus mehreren nicht zusammenhängenden Speicherblöcken in einer einzigen Transaktion zu lesen oder zu schreiben.
- **PCI Transport:** Die spezifische Schicht innerhalb des VirtIO-Protokolls, welche die Kommunikation zwischen dem Gast-System und den virtuellen Geräten über den PCI-Bus (Peripheral Component Interconnect) abwickelt.
- **HAL (Hardware Abstraction Layer):** Eine Softwareschicht, die hardwarenahe Funktionen abstrahiert, um dem darüberliegenden Treiber (hier der `virtio-drivers` Crate) eine einheitliche Schnittstelle unabhängig von der konkreten Plattform zu bieten.
- **MMIO (Memory-Mapped I/O):** Eine Methode zur Kommunikation mit Hardware, bei der Gerätereister in den Adressraum des Arbeitsspeichers eingeblendet werden, sodass die CPU sie wie normalen Speicher lesen und beschreiben kann.
- **„in flight“:** Ein Begriff für Operationen oder Datenpakete, die bereits angestoßen, aber noch nicht vollständig vom Empfänger oder der Hardware verarbeitet und quittiert wurden.
- **BAR (Base Address Register):** Ein Konfigurationsregister im PCI-Konfigurationsraum, das definiert, an welcher Adresse im Speicher oder I/O-Bereich die Register eines Geräts für die CPU erreichbar sind.
- **PCM (Pulse Code Modulation):** Ein Verfahren zur digitalen Repräsentation analoger Signale, das im VirtIO-Sound-Treiber verwendet wird, um unkomprimierte Audiodaten (Samples) zu übertragen.
- **VM Exit:** Ein erzwungener Wechsel der Kontrolle vom Gast-Betriebssystem zurück an den Hypervisor. Dieser Kontextwechsel tritt bei privilegierten Operationen oder Hardwarezugriffen auf und gilt aufgrund des hohen Rechenaufwands als leistungskritisch.

Kapitel 2

Technische Grundlagen

2.1 Virtualisierung und paravirtualisierte Geräte

Virtuelle Maschinen (VMs) ermöglichen das Ausführen eines Betriebssystems in einer isolierten Umgebung, ohne dass dieses direkt auf physische Hardware zugreift. Aus Sicht des Gasts erscheint die Umgebung wie ein vollständiger Rechner. Tatsächlich werden CPU-Ausführung und Gerätezugriffe durch eine Virtualisierungsschicht kontrolliert und bereitgestellt.

2.1.1 Host, Gast und Hypervisor

Die physische Maschine wird als Host bezeichnet, das darin ausgeführte Betriebssystem als Gast. Die Virtualisierungskomponente, die den Gast ausführt und die virtuelle Hardware bereitstellt, wird meist als Hypervisor bzw. Virtual Machine Monitor (VMM) beschrieben, für diese Arbeit ist das QEMU. Im Linux-Umfeld stellt KVM (Kernel-based Virtual Machine) die Hypervisor-Funktionalität im Kernel bereit. VMs werden dabei als Prozesse im Host-System umgesetzt und erhalten „privatisierte“ virtuelle Hardware.

In der Praxis kommen unterschiedliche Host-Umgebungen vor [3]:

- In einer Ubuntu-Umgebung ist KVM eine typische Basis für Virtualisierung.
- WSL2 nutzt Virtualisierung und basiert auf einem Subset der Hyper-V-Architektur.

Für diese Arbeit ist jedoch wichtig, dass das **Bare-Metal**-Betriebssystem (D3OS) als Gast in einer VM läuft, und die I/O-Geräte werden durch den VMM bereitgestellt.

2.1.2 Gerätemodelle

Damit ein Gast I/O durchführen kann (z. B. Grafik, Audio), muss der VMM entsprechende Geräte anbieten. Grundsätzlich lassen sich drei Ansätze unterscheiden [4]:

1. **Emulation:** klassischer Hardware: Der VMM emuliert bekannte Geräte. Das ist kompatibel, kann aber einen höheren Overhead verursachen, da Gerätefunktionen vollständig nachgebildet werden müssen.

2. **Passthrough:** Ein reales Gerät wird direkt an den Gast durchgereicht. Dies bietet hohe Nähe zur Hardware, aber höhere Anforderungen an Plattform und Setup und für viele Entwicklungsumgebungen unpraktisch.
3. **Paravirtualisierung:** Gast und VMM nutzen ein speziell für Virtualisierung entworfenes Geräteinterface, um Emulationsaufwand zu reduzieren.

Für diese Arbeit ist der dritte Ansatz zentral: VirtIO-Geräte sind paravirtualisierte Geräte, die explizit darauf ausgelegt sind, effizient virtualisiert bzw. emuliert zu werden und daher als empfohlene Gerätemodelle in VMs gelten.

2.2 VirtIO-Architektur

VirtIO ist ein standardisiertes Modell für paravirtualisierte Geräte, das die Aufgaben zwischen Gast und Host bewusst trennt. Im Gast läuft der Treiber (Frontend) als Teil des Betriebssystems, im Host/Hypervisor läuft die Geräteimplementierung (Backend). Beide Seiten kommunizieren nicht über „klassische“ emulierte Register- und Datenpfade historischer Hardware, sondern über ein vereinfachtes, für Virtualisierung optimiertes Protokoll.

Der zentrale Kommunikationsmechanismus sind dabei Virtqueues. Das sind im Gast-Speicher abgelegte Warteschlangenstrukturen, auf die Treiber und Gerät zugreifen. Der Treiber hinterlegt Anfragen in Form von Speicherpuffern, die anschließend vom Backend verarbeitet werden. Die Rückmeldung der Ergebnisse erfolgt über dieselben Datenstrukturen, was die Effizienz häufiger Datentransfers maßgeblich steigert [5].

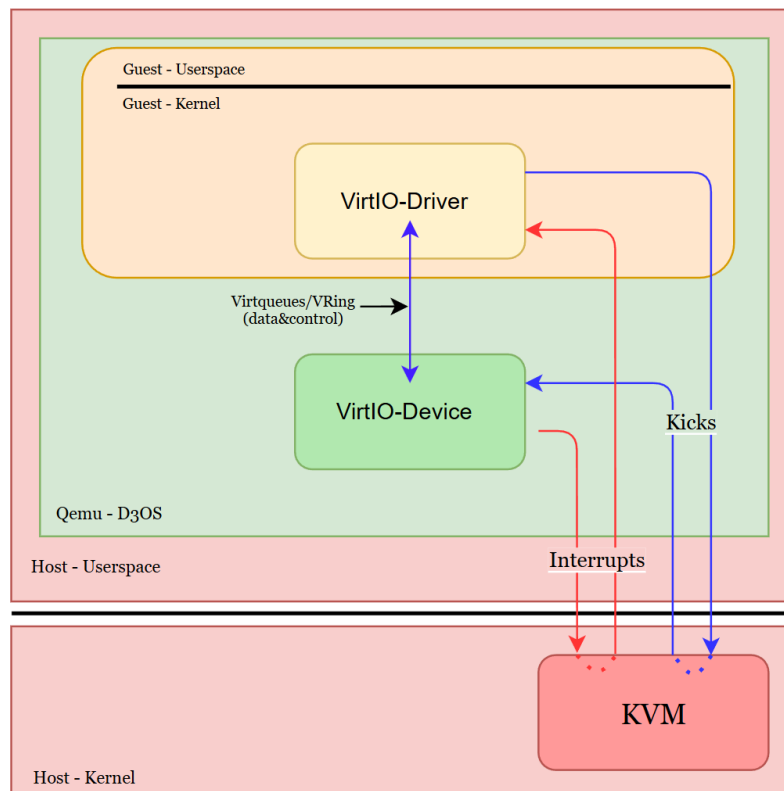


Abbildung 2.1: Interaktion zwischen VirtIO-Treiber und Gerät (eigene Darstellung nach [6])

2.2.1 VirtIO-Geräte

VirtIO-Geräte sind virtuelle Peripheriegeräte, die eine virtuelle Maschine (z. B. in QEMU) dem Gastbetriebssystem zur Verfügung stellt. Statt eine konkrete reale Hardware nachzuahmen, bieten diese Geräte eine standardisierte VirtIO-Schnittstelle. Damit das Betriebssystem diese Geräte nutzen kann, braucht es passende VirtIO-Treiber, die das Gerät initialisieren, konfigurieren und I/O-Anfragen über Virtqueues abwickeln.

Das VirtIO-Modell ist so definiert, dass es über unterschiedliche „Transports“ bereitgestellt werden kann. Die Transports, welche unterstützt werden sind:

- VirtIO over MMIO: Das Gerät stellt Registerbereiche im Memory-Mapped-I/O bereit, der Treiber greift über MMIO auf Status/Features/Queue-Setup zu.
- VirtIO over PCI: Das Gerät erscheint als PCI-Gerät und wird über PCI-Mechanismen entdeckt/konfiguriert (häufig bei x86/QEMU).
- VirtIO over CCW (s390): Spezielle Transportform für IBM s390-Umgebungen.

Bei der in dieser Arbeit behandelten Implementation wird VirtIO over PCI verwendet, da es sich bei D3OS um ein x86 Betriebssystem handelt, welches in QEMU läuft.

Unabhängig vom Transport besitzen VirtIO-Geräte wiederkehrende Kernbestandteile, die jeder Treiber bedienen muss [7]:

Gerätestatus (Device Status)

Das Statusfeld bildet den Lebenszyklus der Initialisierung ab. Der Treiber bestätigt zunächst, dass er das Gerät erkannt hat, signalisiert dann, dass ein Treiber geladen ist, handelt Features aus und markiert am Ende, dass das Gerät betriebsbereit ist. Tritt ein Fehler auf, können sowohl der Treiber als auch das Gerät einen Fehlerzustand signalisieren. In einem solchen Fall ist in der Regel ein Zurücksetzen der virtuellen Geräteschnittstelle erforderlich, um alle Register und Warteschlangen in einen sauberen Ausgangszustand zu versetzen. Praktisch ist der Status damit ein Handshake, der verhindert, dass Gerät und Treiber in falscher Reihenfolge arbeiten.

Feature-Bits (Feature Negotiation)

VirtIO-Geräte veröffentlichen eine Menge an Fähigkeiten als Bitmaske. Der Treiber wählt daraus die Teilmenge, die er unterstützt und aktivieren möchte. Diese Aushandlung stellt die Kompatibilität zwischen verschiedenen Implementierungsständen sicher. Sie ermöglicht es, Geräte um neue Funktionen zu erweitern, ohne die Lauffähigkeit älterer Treiber zu beeinträchtigen, sofern diese lediglich die ihnen bekannten Feature-Bits verarbeiten.

Benachrichtigungen (Notifications)

Sobald der Treiber neue Arbeit in eine Virtqueue gestellt hat, muss das Gerät davon erfahren, z. B. über ein „Notify/Kick“. Umgekehrt signalisiert das Gerät dem Treiber erledigte Arbeit typischerweise über Interrupts (oder der Treiber pollt). In einem Kernel-Design wird häufig der Interrupt nur als „Wecksignal“ genutzt, während die eigentliche Verarbeitung außerhalb des Interrupt-Kontextes erfolgt.

Gerätekonfigurationsbereich (Device Configuration)

Zusätzlich zu generischen Strukturen besitzt jedes VirtIO-Gerät einen gerätespezifischen Konfigurationsbereich. Dort stehen Parameter, die für die Nutzung des Geräts relevant sind wie z.B. Eigenschaften der virtuellen Hardware. Treiber lesen diese Werte typischerweise bei der Initialisierung und bei Konfigurationsänderungen.

Virtqueues

Virtqueues sind das Transportmittel für I/O-Anfragen und Antworten. Ein Gerät kann mehrere Queues besitzen (z. B. getrennte Queues für Control und Datenpfade). Die genaue Anzahl und Bedeutung hängt vom Gerätetyp ab.

2.2.2 Virtqueues

Das zentrale Element der Datenübertragung zwischen Gast (Treiber) und Host (Device) ist die Virtqueue. Um den Overhead von Kontextwechseln bei der Virtualisierung zu minimieren, setzt VirtIO auf Shared Memory in Kombination mit Direct Memory Access (DMA).

Anstatt Daten ineffizient Register für Register durch die CPU zu schleusen, nutzen Treiber und Gerät gemeinsam zugreifbare Speicherstrukturen, um lediglich Metadaten (Deskriptoren) auszutauschen. Diese Deskriptoren verweisen auf physische Speicheradressen im Gastsystem. Das Gerät verarbeitet die eigentlichen Nutzdaten anschließend selbstständig per DMA-Zugriff, ohne dass die Gast-CPU für den Datentransfer belastet wird.

Die VirtIO-Spezifikation kennt dabei zwei Formate für Virtqueues:

- **Split Virtqueue:** klassisches, weit verbreitetes Layout mit drei getrennten Bereichen (Descriptor Table, Avail Ring, Used Ring). Es ist vergleichsweise einfach zu implementieren und zu debuggen.
- **Packed Virtqueue:** moderneres Layout (VirtIO 1.1), das die zuvor getrennten Strukturen stärker zusammenführt und dadurch weniger Speicherzugriffe, weniger Cache-Contention und (bei echter Hardware) weniger PCI-Transaktionen verursacht.

Da der in dieser Arbeit eingebundene Treiber-Stack auf Split Queues basiert, wird im Folgenden zunächst dieses Format detailliert erläutert. Packed Queues werden anschließend kurz eingeordnet.

Split Virtqueue

Eine Split Virtqueue besteht aus drei Speicherbereichen, die im Gast-Speicher liegen und von Treiber und Gerät gemeinsam genutzt werden. Eine wichtige Eigenschaft ist dabei, dass jeder dieser Bereiche im Normalfall nur von einer Seite beschreibbar ist (Treiber oder Gerät), was Datenrennen reduziert und das Design vereinfacht.

Die drei Bestandteile sind [8]:

1. Descriptor Table (Descriptor Area)
2. Available Ring (Driver Area/Avail)

3. Used Ring (Device Area/Used)

Zusätzlich hat jede Queue eine Queue Size (maximale Anzahl gleichzeitig „in flight“ befindlicher Puffer/Requests). Außerdem müssen die einzelnen Teile physisch zusammenhängend im Gast-Speicher liegen und Alignment-Anforderungen erfüllen [5].

Descriptor Table

Die Deskriptortabelle fungiert als das Inhaltsverzeichnis für die eigentlichen Datenpuffer. Sie ist ein Array von Deskriptoren, wobei jeder Eintrag einen Speicherbereich im Adressraum des Gasts beschreibt. Ein einzelner Deskriptor besteht dabei aus vier Kernelementen:

1. Address: Die physische 64-Bit-Startadresse des Puffers (GPA - Guest Physical Address).
2. Length: Die Größe des Puffers in Bytes.
3. Flags: Steuerflags, die z.B. die Zugriffsrichtung regeln (`VIRTIO_DESC_F_WRITE`). Ist dieses Flag gesetzt, darf das Gerät in den Puffer schreiben (z. B. für den Empfang von Netzwerkpaketen), andernfalls ist der Puffer für das Gerät "Read-Only" (z. B. beim Senden).
4. Next: Ein Index-Verweis auf einen weiteren Deskriptor.

Das Next-Feld und das korrespondierende Flag `VIRTIO_DESC_F_WRITE` sind essenziell für das Chaining-Konzept. Da I/O-Anfragen oft aus mehreren Fragmenten bestehen (z. B. einem Protocol-Header und den eigentlichen Nutzdaten), erlaubt VirtIO das Verketteten mehrerer Deskriptoren zu einer logischen Einheit. Für das Gerät erscheint eine solche Kette wie ein einziger logischer Puffer, obwohl die Daten physisch fragmentiert im RAM liegen können. Dies reduziert die Notwendigkeit für teure memcpy-Operationen im Gastsystem drastisch.

Available Ring

Während die Deskriptortabelle nur die Speicherbereiche definiert, regelt der Available Ring den Besitzübergang dieser Puffer vom Treiber an das Gerät. Er ist der Mechanismus, mit dem der Treiber dem Gerät mitteilt, dass der Puffer vorbereitet wurde und nun vom Gerät verarbeitet werden darf.

Der Ring ist als Ringpuffer implementiert, der Indizes enthält, die auf die Köpfe (Heads) von Deskriptorketten in der Deskriptortabelle verweisen. Der Ablauf ist dabei wie folgt:

1. Der Treiber füllt freie Deskriptoren mit Pufferinformationen.
2. Er schreibt den Index des ersten Deskriptors (Head Index) in das nächste freie Slot des Available Rings.
3. Er inkrementiert den idx-Zeiger des Rings, um dem Gerät zu signalisieren, dass neue Arbeit vorliegt.

Ein entscheidender Aspekt ist hier die Trennung von Daten und Signalisierung. Das bloße Eintragen in den Ring löst noch keine Verarbeitung aus. Erst durch einen expliziten „Kick“ (in der Regel ein PIO-Write auf ein Doorbell-Register) wird der Hypervisor/das Gerät benachrichtigt. Um die Anzahl dieser teuren VM-Exits zu reduzieren, unterstützt VirtIO Mechanismen zur Event-Unterdrückung (`VIRTIO_F_EVENT_IDX`), bei denen das Gerät dem Treiber mitteilen kann, erst nach einer bestimmten Anzahl neuer Aufträge wieder geweckt zu werden.

Used Ring

Der Used Ring ist das Gegenstück zum Available Ring und wird primär vom Gerät beschrieben. Hier trägt das Gerät die Indizes der Deskriptorketten ein, deren Verarbeitung abgeschlossen ist. Sobald ein Eintrag im Used Ring erscheint, geht der Besitz des zugehörigen Speichers wieder an den Treiber zurück.

Ein Eintrag im Used Ring enthält neben dem Index der verarbeiteten Kette auch ein `len`-Feld. Dieses gibt an, wie viele Bytes das Gerät tatsächlich in den Puffer geschrieben hat. Dies ist besonders bei variablen Datenmengen (wie Netzwerkpaketen) wichtig ist, da sie oft von der ursprünglichen Puffergröße abweichen.

Nach dem Aktualisieren des Used Rings sendet das Gerät typischerweise einen Interrupt an den Gast, um den Treiber über die Fertigstellung zu informieren. Auch hier greifen Mechanismen zur Interrupt-Drosselung, um die CPU-Last im Gastsystem zu schonen, wenn das Aufkommen an I/O-Events sehr hoch ist.



Abbildung 2.2: Der vollständige Zyklus eines Puffers in einer Split Virtqueue [5]

Packed Virtqueues

Während das in dieser Arbeit verwendete Split-Layout (Descriptor Table, Available Ring, Used Ring) durch seine robuste Trennung der Speicherbereiche besticht, führt es auf modernen Hardwarearchitekturen zu einem erhöhten Verwaltungsaufwand. Da die drei Bereiche im Speicher verstreut liegen (keine Cache-Lokalität), muss die CPU bzw. das Gerät für eine einzelne I/O-Operation oft auf mehrere, weit voneinander entfernte Cache-Lines zugreifen. Dies erhöht die Anzahl der PCIe-Transaktionen und kann zu sogenannten Cache-Bounces führen [9].

Um diesen Overhead zu adressieren, führt der VirtIO 1.1-Standard das Konzept der Packed Virtqueues ein. Im Gegensatz zum Split-Modell werden hierbei die drei getrennten Strukturen in einer einzigen, linearen Ringstruktur zusammengeführt.

Die Funktionsweise unterscheidet sich dabei grundlegend in der Zustandsverwaltung:

- Einheitliche Deskriptoren: Jeder Eintrag im Ring enthält sowohl die Daten-Metadaten (Adresse, Länge) als auch den Status des Eintrags. Es gibt keine separaten Indizes für Available oder Used.
- Wrap-Counter-Logik: Anstelle eines stetig wachsenden Index wird ein Wrap Counter (Umlaufzähler) verwendet, der als binäres Flag in jedem Deskriptor realisiert ist [9]. Bei jedem Durchlauf des Rings „flippt“ dieses Bit. Treiber und Gerät erkennen anhand dieses Bits, ob ein Eintrag neu bereitgestellt wurde oder noch veraltet ist, ohne auf externe Index-Variablen zugreifen zu müssen.

Diese kompaktere Struktur bietet signifikante Vorteile für die Performance, da Treiber und Gerät sequenziell auf denselben Speicherbereich zugreifen, wird die Cache-Nutzung optimiert (bessere Cache-Lokalität) und die Anzahl der notwendigen Speicherzugriffe reduziert.

2.2.3 Benachrichtigungssteuerung

In der Standard-Implementierung können Treiber und Gerät Benachrichtigungen (Interrupts bzw. Kicks) nur pauschal an- oder abschalten. Dies erfolgt über das Flag `VRING_AVAIL_F_NO_INTERRUPT` im Available Ring sowie `VRING_USED_F_NO_NOTIFY` im Used Ring. Dieser Ansatz ist jedoch oft ineffizient: Schaltet der Treiber Interrupts ab, muss er pollen (Polling), was CPU-Zeit kostet. Lässt er sie an, erzeugt jedes einzelne Paket einen teuren VM-Exit und Kontextwechsel.

Um dieses Problem zu lösen, nutzt virtio-drivers das Feature `VIRTIO_F_EVENT_IDX`. Dieses Feature ersetzt die pauschalen Flags durch eine genauere Steuerung basierend auf Ring-Indizes. Dabei werden zwei zusätzliche 16-Bit-Felder am Ende der Ring-Strukturen genutzt [5].

- **used_event** (im Available Ring): Dieses Feld wird vom Treiber beschrieben und dient der Interrupt-Moderation. Der Treiber hinterlegt hier einen Schwellenwert, der sich auf den Index des Used Rings bezieht. Das Gerät wird durch diesen Mechanismus angewiesen, erst dann einen Interrupt auszulösen, wenn der idx-Zeiger des Used Rings den im `used_event`-Feld spezifizierten Wert erreicht.
- **avail_event** (im Used Ring): Dieses Feld wird vom Gerät verwaltet und steuert die Benachrichtigung durch den Gast (Kick). Das Gerät legt hier fest, wie weit der Treiber den Available Ring füllen darf, bevor eine explizite Benachrichtigung an den Host erforderlich ist. Solange der aktuelle Index des Available Rings den Wert in `avail_event` nicht überschreitet, kann der Treiber weitere Deskriptoren bereitstellen, ohne einen teuren VM-Exit zu provozieren.

Obwohl diese Mechanismen zur Benachrichtigungsunterdrückung laut Spezifikation lediglich als Optimierung dienen, reduzieren sie den Overhead für die Signalisierung in der Praxis signifikant. Durch die Vermeidung unnötiger Kontextwechsel zwischen Gast und Host wird der I/O-Durchsatz gesteigert, da Benachrichtigungen effizient gebündelt (Batching) verarbeitet werden können [8, Abschnitt 2.7.7].

2.3 VirtIO-GPU

Das VirtIO-GPU-Gerät fungiert als paravirtualisierter Grafikadapter, der dem Gastsystem ermöglicht, grafische Ausgaben zu erzeugen, ohne auf die komplexe Register-Emulation physischer Grafikkarten angewiesen zu sein. In seiner Grundkonfiguration stellt das Gerät einen virtuellen Framebuffer bereit, dessen Inhalte vom Hypervisor (z. B. QEMU) in einem Fenster auf dem Hostsystem dargestellt werden.

Architektur und Kommunikationskanäle

Im Gegensatz zu traditionellen Grafikgeräten, die primär über Memory-Mapped I/O (MMIO) Register gesteuert werden, basiert die Steuerung der VirtIO-GPU auf dem Austausch von Kommandostrukturen über Virtqueues. Die Spezifikation definiert hierfür zwei primäre Warteschlangen [8, Abschnitt 5.7]:

1. **Control Queue** (Queue 0): Über diesen Kanal werden sämtliche Steuerbefehle gesendet, die den Zustand des Grafikgeräts verändern. Dazu gehören das Erstellen von Ressourcen, das Zuweisen von Speicherbereichen sowie Signalisierungen für Bildschirmaktualisierungen.
2. **Cursor Queue** (Queue 1): Diese optionale Queue dient der separaten Steuerung eines Hardware-Mauszeigers. Da diese Funktionalität für die grundlegende Bildausgabe nicht zwingend erforderlich ist, wird sie in vielen Basis-Implementationen vernachlässigt.

Ressourcen-Management und Framebuffer

Ein zentrales Konzept der VirtIO-GPU ist die Trennung zwischen der logischen Ressource und dem physischen Backing Store. Eine Ressource ist dabei lediglich ein abstraktes Objekt (identifiziert durch eine Resource-ID), das Metadaten wie Breite, Höhe und Pixelformat (z. B. RGBA, 32-Bit pro Pixel) beschreibt. Damit diese Ressource tatsächlich Daten aufnehmen kann, muss ihr explizit Gast-physischer Speicher zugewiesen werden (Backing). Dieser Speicherbereich fungiert als Framebuffer, in den der Treiber die Farbwerte der einzelnen Pixel schreibt.

Initialisierungsablauf (2D-Modus)

Um eine funktionierende Bildausgabe zu etablieren, folgt der Treiber einem definierten Protokoll, das aus vier wesentlichen Phasen besteht. Diese Befehle werden als Anfragestrukturen (Requests) in die Control Queue gestellt und vom Gerät quittiert [8, Abschnitt 5.7]:

1. Erkennung der Display-Konfiguration (GET_DISPLAY_INFO): Zunächst fragt der Treiber die aktuelle Konfiguration des virtuellen Monitors ab. Das Gerät liefert Informationen über unterstützte Auflösungen oder die aktuelle Fenstergröße des Hypervisors zurück. Dies ermöglicht dem Gast, seine interne Auflösung dynamisch anzupassen.

2. Erstellung der 2D-Ressource (RESOURCE_CREATE_2D): Der Treiber weist das Gerät an, eine neue interne Ressource mit einer eindeutigen ID zu erstellen. Hierbei werden das Format (typischerweise B8G8R8A8_UNORM) sowie die Dimensionen (Breite und Höhe) festgelegt. Zu diesem Zeitpunkt besitzt die Ressource noch keinen Speicherplatz für Bilddaten.
3. Zuweisung des Speichers (RESOURCE_ATTACH_BACKING): Im dritten Schritt verknüpft der Treiber die zuvor erstellte Ressource mit einem physischen Speicherbereich im RAM des Gasts. Der Treiber übergibt hierbei eine Liste von Speicheradressen (Scatter-Gather-Liste), die als Backing Store dienen. Ab diesem Moment kann das Gerät per DMA (Direct Memory Access) auf die Pixeldaten zugreifen, die der Treiber in diesen Speicherbereich schreibt.
4. Datenübertragung (TRANSFER_TO_HOST_2D): Nachdem der Treiber Pixeldaten in den lokalen Backing-Store geschrieben hat, sendet er diesen Befehl. Er weist das Gerät an, die Daten aus dem Gast-Speicher in die interne Ressource des Hosts (z. B. in den Videospeicher der Host-Grafikkarte) zu kopieren.
5. Aktivierung der Ausgabe (SET_SCANOUT): Abschließend wird die Verbindung zwischen der Ressource und dem virtuellen Monitorausgang (Scanout) hergestellt. Durch diesen Befehl wird festgelegt, dass der Inhalt der spezifischen Ressource auf dem Bildschirm dargestellt werden soll.

Um Änderungen sichtbar zu machen, muss jedoch zwingend ein Flush-Befehl (RESOURCE_FLUSH) gesendet werden, der den Hypervisor anweist, die geänderten Daten aus dem Gastspeicher zu lesen und das Host-Fenster zu aktualisieren.

Während dieser 2D-Modus für einfache grafische Oberflächen ausreicht, stößt er bei komplexeren Anwendungen an seine Grenzen: Er erlaubt lediglich den Austausch fertiger Pixeldaten (Dumb Framebuffer), bietet jedoch keine Möglichkeit, die Rechenleistung der Host-GPU für Renderoperationen zu nutzen. Um hardwarebeschleunigtes Rendering zu ermöglichen, erweitert der Standard die Spezifikation um 3D-Funktionalitäten. Die technische Basis hierfür bildet das VirGL-Protokoll, dessen Funktionsweise im folgenden Abschnitt erläutert wird.

2.3.1 3D-Beschleunigung und VirGL

Die Erweiterung des VirtIO-GPU-Standards um 3D-Fähigkeiten (**virgl**) verfolgt das Ziel, OpenGL oder Vulkan-Befehle vom Gast an den Host durchzureichen, um dessen Hardwarebeschleunigung zu nutzen.

Insbesondere für ein eigenständiges Bare-Metal-Betriebssystem wie D3OS stellt dies eine signifikante Hürde dar. Etablierte Systeme (wie Linux) greifen auf komplexe Grafik-Stacks (z. B. Mesa 3D) zurück, die Millionen Zeilen Code umfassen, um High-Level-APIs wie OpenGL oder Vulkan bereitzustellen. Da die Implementierung eines vergleichbaren, vollständigen 3D-Stacks, inklusive Shader, Compiler, Speicherverwaltung und State-Tracker, den Rahmen dieser Bachelorarbeit überschreiten würde, wird in dieser Arbeit kein vollständiger 3D-Grafikstack realisiert. Der Fokus liegt stattdessen auf der Enabling-Technologie: Der Implementierung des Transportmechanismus, der es dem D3OS-Kernel erlaubt, grundlegende 3D-Befehlsketten zu konstruieren und erfolgreich an den Host zu

übertragen.

Um dies hardwareunabhängig zu ermöglichen, definiert VirGL eine virtuelle 3D-GPU mit einem standardisierten Befehlssatz. Da der Gast nicht wissen kann, welche physische Grafikkarte im Host verbaut ist (z. B. NVIDIA, AMD oder Intel), kann er keine herstelllerspezifischen Treiber verwenden. VirGL fungiert hier als Abstraktionsschicht, die stark an die Architektur von Gallium3D (einem Treiber-Framework des Linux-Grafikstacks) angelehnt ist [10].

Funktionsprinzip: Der Gallium-Passthrough Der Datenfluss bei der 3D-Beschleunigung unterscheidet sich fundamental vom 2D-Modus. Anstatt Pixel zu kopieren, serialisiert der Treiber im Gastsystem Grafikbefehle (z. B. „Setze Viewport“, „Lade Shader“, „Zeichne Dreieck“) in einen binären Datenstrom.

1. Gast (Frontend): Der Treiber kodiert High-Level-Grafikoperationen in das VirGL-Protokoll. Diese Befehle werden in einen Speicherpuffer geschrieben (Command Buffer).
2. Transport: Über die Virtqueue (speziell über den Befehl SUBMIT_3D) wird dieser Puffer an den Host übergeben.
3. Host (Backend): Auf der Host-Seite läuft eine Komponente (in QEMU typischerweise virglrenderer), die diesen Bytecode entgegennimmt [11]. Sie analysiert die Befehle und übersetzt sie in die native Grafik-API des Hosts (z. B. echtes OpenGL).

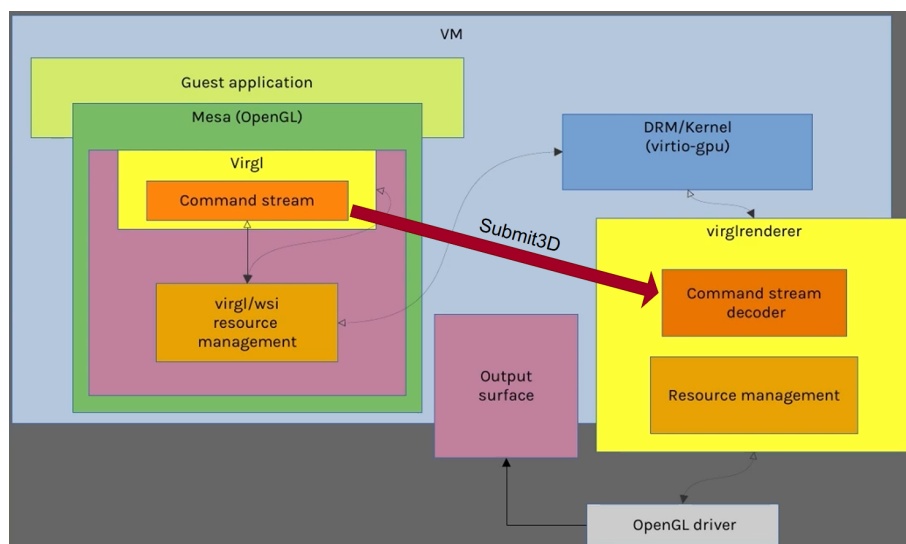


Abbildung 2.3: Referenzarchitektur des VirGL-Renderings im Linux-Grafikstack (Mesa) (eigene Darstellung nach [6])

Die in Abbildung 2.3 dargestellte Architektur visualisiert das Zusammenspiel dieser Komponenten in einem Linux-Referenzsystem. Während dort der Mesa 3D-Stack (grüner Block) die Übersetzung der OpenGL-Befehle übernimmt, verdeutlicht die Grafik den Ansatzpunkt der D3OS-Implementierung. Sie ersetzt den gesamten linken Software-Stack (Gast-Applikation und Mesa).

Der in Kapitel 4 entwickelte Treiber fungiert direkt als Command Stream Encoder (orange Box im Gast). Anstatt High-Level-APIs bereitzustellen, generiert er die binären Befehlsdaten manuell und übergibt sie an den Transportmechanismus. Für den Host (rechte Seite der Abbildung) ist dieser Unterschied transparent, da der generierte Command-Stream dem Protokollstandard entspricht, verarbeitet der **virglrenderer** die Befehle identisch, unabhängig davon, ob sie von einem komplexen Grafikstack oder der direkten Treiberimplementierung stammen.

Um diesen Mechanismus zu nutzen, führt VirtIO-GPU neue Objekttypen und Kommandos ein, die für das Verständnis der späteren Implementation essenziell sind:

- **Virtuelle Kontexte (CTX):** Im Gegensatz zum globalen Zustand des 2D-Modus kapselt VirGL Zustände in Kontexten. Ein Kontext entspricht isolierten OpenGL-Zuständen. Bevor 3D-Befehle gesendet werden können, muss der Treiber mittels `CTX_CREATE` einen Kontext erstellen. Dies ermöglicht es dem Host, mehrere grafische Anwendungen verschiedener VMs sauber zu trennen.
- **3D-Ressourcen:** Ressourcen im 3D-Modus (`RESOURCE_CREATE_3D`) sind mächtiger als ihre 2D-Pendants. Sie besitzen Attribute wie Bind-Flags, die definieren, wie die Ressource genutzt wird (z. B. als Vertex-Buffer, Index-Buffer oder Render-Target). Diese Ressourcen müssen explizit an einen Kontext gebunden werden (`CTX_ATTACH_RESOURCE`), bevor die virtuelle GPU darauf zugreifen kann.
- **Command Submission:** Ein zentraler Bestandteil der 3D-Kommunikation ist die Übermittlung von Befehlen, der Befehl `VIRTIO_GPU_CMD_SUBMIT_3D` überträgt dabei den Command-Stream an den Host. Da Rendering asynchron erfolgt, ist hierbei die Synchronisation entscheidend. Mithilfe von Fences kann der Treiber markieren, wann eine bestimmte Befehlsfolge vom Host abgearbeitet wurde, um Race Conditions im Speicherzugriff zu vermeiden.

Durch diese Architektur wird VirtIO-GPU von einem einfachen Framebuffer-Gerät zu einer vollwertigen, programmierbaren Grafikpipeline erweitert, die dem Gastsystem erlaubt, komplexe 3D-Szenen hardwarebeschleunigt zu rendern.

2.4 VirtIO-Sound

Das VirtIO-Sound-Gerät (`virtio-snd`) stellt eine paravirtualisierte Soundkarte bereit, die sowohl die Wiedergabe (Playback) als auch die Aufnahme (Capture) von Audiodaten ermöglicht. Im Gegensatz zu älteren emulierten Standards (wie AC'97 oder Intel HDA), die oft komplexe Register-Interfaces besitzen, setzt VirtIO-Sound konsequent auf den nachrichtenbasierten Austausch über Virtqueues.

2.4.1 Architektur und Warteschlangen

Die Spezifikation definiert für das Sound-Gerät vier spezialisierte Warteschlangen, die unterschiedliche Aspekte der Audiosteuerung und Datenübertragung abdecken [8, Abschnitt 5.14]:

1. **Control Queue** (Queue 0): Über diesen Kanal werden administrative Aufgaben abgewickelt. Der Treiber nutzt ihn, um Informationen über verfügbare Anschlüsse (Jacks) und Datenströme (Streams) abzufragen oder um Hardware-Parameter wie die Sample-Rate und das Datenformat zu konfigurieren. Auch Zustandswechsel (Start/Stop von Streams) werden hier initiiert.
2. **Event Queue** (Queue 1): Diese Queue dient dem asynchronen Empfang von Benachrichtigungen. Das Gerät informiert den Treiber hierüber beispielsweise, wenn ein Kopfhörer eingesteckt wurde (Jack-Remapping) oder wenn ein abgespielter Audioabschnitt (Period) verarbeitet wurde. Da Audioverarbeitung zeitkritisch ist, ermöglicht diese Trennung von der Control-Queue eine effiziente Ereignisbehandlung.
3. **TX Queue** (Transmit/Queue 2): Dies ist der Datenkanal für die Wiedergabe. Der Treiber stellt Puffer mit PCM-Audiodaten (Pulse Code Modulation) in diese Queue. Das Gerät entnimmt die Daten und leitet sie an das Audio-Backend des Hosts weiter.
4. **RX Queue** (Receive/Queue 3): Dies ist der Datenkanal für die Aufnahme. Der Treiber stellt hier leere Puffer bereit, die vom Gerät mit Audiodaten (z. B. vom Mikrophon) gefüllt werden.

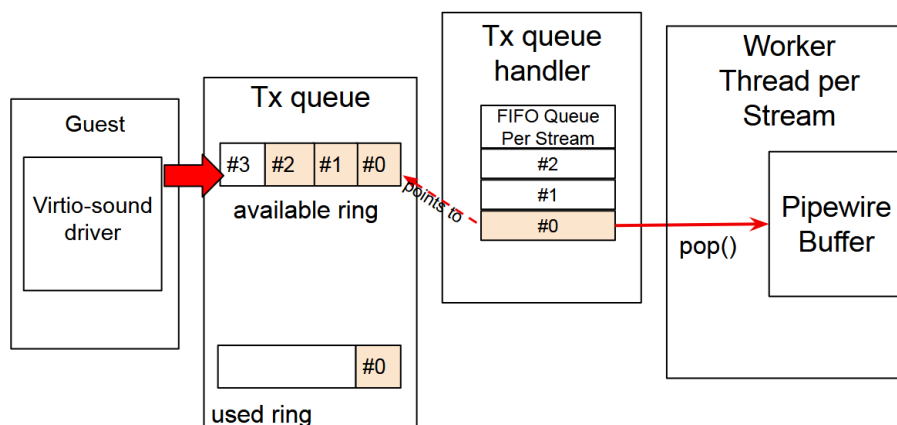


Abbildung 2.4: Datenfluss der VirtIO-Sound TX-Queue (Playback) [12]

Die in Abbildung 2.4 dargestellte Architektur visualisiert den Datenfluss innerhalb der TX Queue während der Audiowiedergabe. Auf der linken Seite platziert der VirtIO-Sound-Treiber des Gastsystems Audiodaten in Form von einzelnen Perioden (hier nummeriert als 0 bis 3) in den Available Ring. Dieser Ring fungiert als geteilter Speicherbereich, über den der Treiber neue Aufgaben ankündigt.

Auf der Host-Seite (rechts) überwacht ein Queue Handler diesen Ring. Sobald neue Puffer verfügbar sind, im Diagramm exemplarisch Puffer 0, werden diese entnommen (pop) und verarbeitet. Das Diagramm zeigt hierbei Pipewire als das Audio-Backend des Hosts. Dies repräsentiert eine typische Konfiguration auf modernen Linux-Hostsystemen (wie dem in der Evaluation genutzten Ubuntu 25.10), bei denen der Hypervisor (QEMU) die Audiodaten aus dem Virtqueue direkt an den Pipewire-Soundserver des Hosts weiterleitet, um die Ausgabe zu realisieren.

Logische Komponenten: Jacks und Streams

Das Gerätemodell abstrahiert die Audio-Hardware in zwei Hauptkomponenten:

- **Jacks:** Repräsentieren physische Anschlüsse (z. B. Mikrofonbuchse, Line-Out). Sie besitzen Eigenschaften wie Verbindungsstatus und Pin-Belegung.
- **PCM Streams:** Repräsentieren die logischen Datenflüsse für Input oder Output. Ein Stream ist durch Parameter wie Kanalanzahl (Mono/Stereo), Format (z. B. 16-Bit Signed Integer) und Frequenz (Sample Rate) definiert.

Pufferung und Perioden

Eine Besonderheit der Audioverarbeitung ist die Notwendigkeit eines konstanten Datenflusses, um Aussetzer (Glitches) zu vermeiden. VirtIO-Sound unterteilt den Audiopuffer dazu in logische Abschnitte, sogenannte Perioden. Anstatt einen einzigen riesigen Puffer zu übergeben, zerlegt der Treiber den Datenstrom in periodengroße Blöcke (Chunks). Das Gerät verarbeitet diese Blöcke sequenziell und sendet nach Abschluss einer Periode optional eine Benachrichtigung (Elapsed Period Message) über die Event-Queue. Dies erlaubt dem Treiber, den Füllstand des Puffers präzise zu überwachen und rechtzeitig neue Daten nachzuliefern (Double- oder Ring-Buffering).

Kapitel 3

Architektur und Einbindung der VirtIO-Treiber

3.1 Integrationskonzept und Anforderungen

Das Integrationskonzept dieser Arbeit basiert auf der Einbindung der Rust-Crate `virtio-drivers` als wiederverwendbare Treiberbasis. Die Crate kapselt dabei die VirtIO-spezifische Treiberlogik, während systemabhängige Aufgaben gezielt vom Betriebssystem bereitgestellt werden. Dadurch wird die Treiberintegration so gestaltet, dass zusätzliche VirtIO-Geräte später konsistent aktiviert und genutzt werden können, ohne für jedes Gerät eine vollständig eigenständige Implementierung entwickeln zu müssen.

Die Kopplung zwischen Betriebssystem und Treiberbasis erfolgt über eine Hardware-Abstraktionsschicht (**HAL**). Diese Schicht stellt die für die Crate notwendigen Plattformfunktionen bereit und bildet sie auf vorhandene Kernel-Mechanismen ab. Dazu gehören insbesondere die Bereitstellung DMA-fähiger Speicherbereiche, die benötigten Adressübersetzungen sowie der Zugriff auf die vom VirtIO-Transport bereitgestellten Geräteregisterbereiche. Damit ergeben sich klare Anforderungen an die Integration: Speicherbereiche, die von VirtIO-Geräten genutzt werden, müssen in geeigneter Form allokiert, gemappt und über ihre Lebensdauer hinweg gültig gehalten werden. Zudem muss die HAL so implementiert werden, dass die von `virtio-drivers` erwarteten Sicherheitsinvarianten eingehalten werden, da Teile der Anbindung im `unsafe`-Kontext stattfinden [13].

Ein wesentliches architektonisches Detail ist dabei die Art der Einbindung. Anstatt die Crate als unveränderliche Abhängigkeit aus der Registry zu beziehen, wird sie in der `Cargo.toml` des Kernels (`\os\kernel\Cargo.toml`) über eine lokale Pfadreferenz eingebunden:

	Cargo.toml	TOML
1	[dependencies]	
2	virtio = { package = "virtio-drivers", path = "../library/virtio-drivers" }	

Dies ermöglicht direkte Modifikationen am Quellcode der Treiberbibliothek. Dieser Zugriff ist essenziell, um später (siehe Kapitel 4) fehlende Funktionalitäten wie die VirGL-Unterstützung direkt in den internen Modulen (`gpu.rs`) nachrüsten zu können, ohne die Treiberarchitektur zu umgehen.

3.2 DMA-Speicherbereitstellung

Dieser Kapitelabschnitt basiert auf Änderungen an `\os\kernel\src\dma\mod.rs`, da der hierfür relevante Code dort verortet ist. Die in dieser Datei implementierte DMA-Schnittstelle wurde unverändert aus der Arbeit von Nikita Elrich übernommen und dient in dieser Arbeit als bestehende Grundlage für die Anbindung von virtio-drivers.

Ziel der DMA-Abstraktion ist es, dem Treiber bzw. der eingebundenen Treiber-Crate zusammenhängende Speicherbereiche bereitzustellen, die vom VirtIO-Gerät direkt adressiert werden können. Diese Speicherbereiche werden insbesondere für gemeinsam genutzte Datenstrukturen wie Virtqueues sowie für Ein- und Ausgabepuffer benötigt.

Der nachfolgende Quelltext zeigt den Aufbau der Dma-Struktur. Sie hält die physikalische Startadresse `paddr`, einen nicht-null virtuellen Zeiger `vaddr` sowie Verwaltungsinformationen zur Größe.

```
mod.rs Rust
1  /// A DMA buffer allocated in physical memory and mapped into virtual address
   space.
2  #[derive(Debug)]
3  pub struct Dma {
4      paddr: PhysAddr,
5      vaddr: NonNull<u8>,
6      size: usize,
7      pages: usize,
8  }
```

Die Allokation erfolgt seitenbasiert über eine Kernel-interne Frame-Allokation. Anschließend wird für den zugehörigen virtuellen Bereich ein geeignetes Page-Table-Flag-Set gesetzt. In der vorliegenden Implementierung wird der Bereich explizit als `present`, `writable` und `uncached` markiert. Das Deaktivieren des Caches ist dabei eine bewusste Designentscheidung, um den Zugriff eines Geräts auf denselben Speicherbereich nicht durch Cache-Effekte zu verfälschen und damit eine robuste DMA-Nutzung zu ermöglichen.

Ein weiterer zentraler Punkt ist die Adressbildung im Kernel. In D3OS ist der Kernelbereich identitätsabbildend, d. h. physikalische und virtuelle Adresse stimmen überein. Dadurch kann die virtuelle Adresse direkt aus der physikalischen Adresse konstruiert werden, was die Anbindung vereinfacht.

```
mod.rs Rust
1  kernel_process.virtual_address_space.set_flags(
2      pages_range,
3      PageTableFlags::PRESENT | PageTableFlags::WRITABLE | PageTableFlags::NO_CACHE,
4  );
5  // In D3OS, the kernel is identity-mapped: physical = virtual address.
6  let vaddr = NonNull::new(paddr.as_u64() as *mut u8).expect("Invalid virtual
   address");
```

Für die Nutzung in Treiberpfaden stellt `Dma` anschließend einfache Zugriffsmethoden bereit. `paddr()` liefert die physikalische Adresse, während `vaddr(offset)` einen virtuellen Zeiger (mit Bounds-Check) auf eine Teilposition innerhalb des Puffers zurückgibt. Zusätzlich ermöglicht `raw_slice()` die Erzeugung eines zusammenhängenden `u8-Slices` über die gesamte DMA-Region, der sich z. B. für das initiale Befüllen von Puffern oder das lineare Schreiben von Nutzdaten eignet.

3.3 HAL-Anbindung

Dieser Kapitelabschnitt basiert auf der Implementierung der HAL-Schicht in `\os\kernel\src\hal.rs`. Die HAL bildet die Schnittstelle zwischen Kernel und der eingebundenen Treiberbasis virtio-drivers. Sie kapselt betriebssystemspezifische Funktionen, die für den Betrieb der VirtIO-Treiber erforderlich sind, insbesondere die Bereitstellung DMA-fähiger Speicherbereiche sowie die Abbildung zwischen physischen und virtuellen Adressen. Damit stellt sie eine zentrale Voraussetzung dar, um die Treiberlogik der Crate in die Systemumgebung einzubetten.

Das HAL-Trait definiert dabei auch einen Safety-Vertrag für die Anbindung der Treiberbibliothek. Die Korrektheit der bereitgestellten Zeiger, die Lebensdauer von DMA-Puffern sowie Annahmen zur Aliasfreiheit können vom Compiler nicht überprüft werden und müssen daher durch die HAL-Implementierung gewährleistet werden. Entsprechend sind Teile der Schnittstelle als unsafe deklariert [13].

Die Methoden `dma_alloc` und `dma_dealloc` bilden die Schnittstelle zwischen virtio-drivers und der in Abschnitt 3.2 beschriebenen DMA-Speicherbereitstellung. Die HAL-Dokumentation fordert dabei, dass `dma_alloc` zusammenhängende physische Seiten allokiert und nullt und einen `NonNull<u8>` zurückgibt, der gültig, seitenaligned ist und bis zur Freigabe nicht mit anderen Referenzen aliasiert. In der hier verwendeten Implementierung wird dafür die bestehende Dma-Abstraktion genutzt, die den DMA-tauglichen Speicherbereich kapselt.

Der nachfolgende Quelltext zeigt den Kern von `dma_alloc`. Ein Dma-Objekt wird erzeugt, die physische Adresse (`paddr`) und die zugehörige virtuelle Adresse (`vaddr`) werden extrahiert und anschließend wird das Objekt mittels `core::mem::forget` absichtlich nicht gedroppt. Dadurch bleibt der Speicher reserviert und die Freigabe wird auf `dma_dealloc` verlagert. Dieses Vorgehen passt zum Interface von virtio-drivers, da die Crate die spätere Deallokation explizit über `dma_dealloc` anstößt.

hal.rs	Rust
1	<code>fn dma_alloc(pages: usize, _direction: BufferDirection) -> (PhysAddr, NonNull<u8>)</code>
2	<code>{</code>
3	<code> let dma_buffer = Dma::new(pages);</code>
4	<code> let paddr = dma_buffer.paddr().as_u64() as PhysAddr;</code>
5	<code> let vaddr = dma_buffer.vaddr(0);</code>
6	<code></code>
7	<code> core::mem::forget(dma_buffer);</code>
8	<code> (paddr, vaddr)</code>
9	<code>}</code>

Die korrespondierende Freigabe erfolgt über `dma_dealloc`. Gemäß Spezifikation ist diese Methode unsafe, da der Aufrufer garantieren muss, dass es sich genau um den zuvor von `dma_alloc` gelieferten Speicherbereich handelt und dass `pages`, `paddr` und `vaddr` übereinstimmen. In der Implementierung wird aus `paddr` der Start-Frame bestimmt und anschließend der Frame-Bereich über das Frame-Subsystem freigegeben.

```

hal.rs Rust
1 unsafe fn dma_dealloc(paddr: PhysAddr, _vaddr: NonNull<u8>, pages: usize) -> i32 {
2     let start_frame =
3         PhysFrame::from_start_address(x86_64::PhysAddr::new(paddr as
4             u64)).unwrap();
5     let frame_range = x86_64::structures::paging::frame::PhysFrameRange {
6         start: start_frame,
7         end: start_frame + pages as u64,
8     };
9     crate::memory::frames::free(frame_range);
10 }

```

Für den PCI-Transport muss virtio-drivers MMIO-Adressen aus BARs in eine für den Treiber nutzbare virtuelle Adresse übersetzen können. Die HAL-Dokumentation beschreibt hierfür `mmio_phys_to_virt` und erlaubt optional Validierung der Region über `paddr` und `size`. Aufgrund des Identity Mappings ist hier die physische Adresse einfach als virtuelle zu interpretieren.

```

hal.rs Rust
1 unsafe fn mmio_phys_to_virt(paddr: PhysAddr, _size: usize) -> NonNull<u8> {
2     NonNull::new(paddr as *mut u8).unwrap()
3 }

```

Damit erfüllt die HAL-Schicht die zentrale Funktion als Adapter, sie stellt virtio-drivers die betriebssystemspezifischen Primitive zur Verfügung, insbesondere DMA-Allokation, (MMIO-)Adressierung und Speicherfreigabe. Auf dieser Grundlage kann im nächsten Abschnitt die Geräteinitialisierung über PCI sowie die Erzeugung und Bereitstellung konkreter Treiberinstanzen beschrieben werden.

3.4 Geräteinitialisierung

Dieser Kapitelabschnitt basiert auf Änderungen an `\os\kernel\src\boot.rs`, da dort die Erkennung von VirtIO-Geräten über PCI, das Mapping der Gerätespeicherbereiche sowie die initiale Treiberinstanziierung umgesetzt sind. Ziel des Initialisierungspaths ist es, VirtIO-Geräte beim Systemstart zuverlässig zu finden, die für den Zugriff notwendigen Ressourcen bereitzustellen und anschließend konkrete Treiberinstanzen zu erzeugen, die im weiteren Systemverlauf genutzt werden können.

Damit der folgende Code funktioniert und VirtIO-Geräte gefunden werden, müssen entsprechende VirtIO-Geräte in der QEMU-Startkonfiguration aktiviert sein. Andernfalls liefert die PCI-Enumeration zwar weiterhin alle vorhandenen Geräte, es existieren jedoch keine Einträge mit der VirtIO **Vendor-ID** `0x1AF4`. In der verwendeten `Makefile.toml` werden die für diese Arbeit relevanten Geräte daher explizit hinzugefügt, z. B. für die VirtIO-GPU und VirtIO-Sound:

Makefile.toml

TOML

```

1 # VirtIO GPU
2 "-device", "virtio-vga-gl"
3 #VirtIO Sound
4 "-device", "virtio-sound-pci,audiodev=audio0"

```

3.4.1 VirtIO-Erkennung

Die Suche nach VirtIO-Geräten erfolgt über den PCI-Bus anhand der VirtIO Vendor-ID `0x1AF4`. Für jedes gefundene Gerät wird zunächst eine DeviceFunction erzeugt, die den eindeutigen PCI-Ort (Bus/Device/Function) beschreibt.

boot.rs

Rust

```

1 const VIRTIO_VENDOR_ID: u16 = 0x1AF4;
2
3 let virtio_devices = pci_bus.search_by_vendor(VIRTIO_VENDOR_ID);
4 let mut pci_root = PciRoot::new(unsafe { pci_config_space.unsafe_clone() });
5
6 for device_lock in virtio_devices.iter() {
7     let address = device_lock.read().header().address();
8
9     let device_function = DeviceFunction {
10         bus: address.bus(),
11         device: address.device(),
12         function: address.function(),
13     };
14     // ...
15 }

```

Zur technischen Umsetzung der Erkennung wurde die PciBus-Struktur in `\os\kernel\src\device\pci.rs` um die Hilfsmethode `search_by_vendor` erweitert. Diese iteriert über alle beim Systemstart enumerierten PCI-Geräte und filtert diese anhand der Vendor-ID, wodurch eine effiziente Selektion der relevanten VirtIO-Geräte ermöglicht wird.

pci.rs

Rust

```

1 pub fn search_by_vendor(&self, vendor_id: u16) -> Vec<&RwLock<EndpointHeader>> {
2     self.devices
3         .iter()
4         .filter(|device| device.read().header().id(self.config_space()).0 ==
5             vendor_id)
6         .collect()
7 }

```

Damit die instanziierten VirtIO-Treiber anschließend Zugriff auf den PCI-Konfigurationsraum erhalten, muss zudem eine Schnittstellenkompatibilität zwischen dem Kernel-eigenen PCI-Subsystem und der externen VirtIO-Bibliothek hergestellt werden. Hierfür implementiert die Struktur `ConfigurationSpace` den Trait `VirtioConfigAccess`. Diese Implementierung fungiert als Adapter-Schicht, sie übersetzt die adressierungs- und zugriffsspezifischen Typen der VirtIO-Bibliothek (z.B. DeviceFunction) in die internen Datenstrukturen des Betriebssystems (`PciAddress`) und leitet Lese- und Schreibzugriffe an die bestehende `ConfigRegionAccess`-Logik weiter.

```

pci.rs Rust
1 impl VirtioConfigAccess for ConfigurationSpace {
2     fn read_word(&self, device_function:
        virtio::transport::pci::bus::DeviceFunction, register_offset: u8) -> u32 {
3         let address = PciAddress::new(0, device_function.bus,
            device_function.device, device_function.function);
4         unsafe { self.read(address, register_offset as u16) }
5     }
6
7     fn write_word(&mut self, device_function:
        virtio::transport::pci::bus::DeviceFunction, register_offset: u8, data: u32)
8     {
9         let address = PciAddress::new(0, device_function.bus,
            device_function.device, device_function.function);
10        unsafe { self.write(address, register_offset as u16, data) };
11    }
12
13    unsafe fn unsafe_clone(&self) -> Self {
14        self.clone()
15    }
16 }

```

3.4.2 BAR-Mapping

Damit virtio-drivers später über den PCI-Transport auf die relevanten Registerbereiche zugreifen kann, müssen die entsprechenden BAR-Regionen im Kerneladressraum verfügbar sein. Dazu iteriert der Boot-Code über die BAR-Einträge des Geräts und mappt geeignete Speicherbereiche als Device Memory.

Wichtig ist dabei, dass PCI 64-Bit Memory-BARs zwei BAR-Registerplätze belegen (Low- und High-DWORD, z. B. BAR2 und BAR3) [14]. Der nachfolgende BAR-Slot enthält somit keine eigenständige BAR, sondern lediglich das obere 32-Bit-Wort der bereits ausgewerteten 64-Bit-Adresse. Dieser Eintrag wird daher bewusst übersprungen, um zu vermeiden, dass das High-DWORD fälschlich als separate BAR interpretiert wird

```

boot.rs Rust
1 for bar_index in 0..6 {
2     if let Ok(Some(BarInfo::Memory { address, size, .. })) =
3         pci_root.bar_info(device_function, bar_index)
4     {
5         if size == 0 {
6             continue;
7         }
8         if address % PAGE_SIZE as u64 != 0 {
9             warn!("Skipping non-page-aligned 64-Bit Bar{} at {:#x}", bar_index,
                address);
10            continue;
11        }
12        // ...
13    }
14 }

```

Im Rahmen der BAR-Auswertung wird geprüft, ob ein zu mappender Gerätespeicherbereich mit dem bereits durch Multiboot bereitgestellten Framebufferbereich überlappt. Diese Situation tritt in der verwendeten Konfiguration auf, da die virtuelle GPU in QE-

MU über die Geräteoption `-device virtio-vga-gl` eingebunden wird und dadurch der VGA-Framebuffer physikalisch an derselben Basisadresse beginnt wie der Multiboot-Framebuffer. In diesem Fall wird auf ein erneutes Mapping verzichtet, stattdessen wird das vorhandene Framebuffer-Mapping wiederverwendet und um den zusätzlichen Adressbereich erweitert. Dadurch bleibt der Bootvorgang unabhängig von der konkreten Gerätekonfiguration korrekt, also sowohl mit als auch ohne VirtIO-GPU als VGA-Gerät.

```

boot.rs Rust
1  #[inline(always)]
2  const fn overlaps(a_start: u64, a_end: u64, b_start: u64, b_end: u64) -> bool {
3      !(a_end <= b_start || b_end <= a_start)
4  }

boot.rs Rust
1  if overlaps(bar_start, bar_end, fb_start_phys_addr, fb_end_phys_addr) {
2      // Bereits gemappter Multiboot-LFB (virtio-vga BAR0)
3      info!(
4          "BAR{} {:#x}..{:#x} overlaps framebuffer {:#x}..{:#x},
5          bar_index, bar_start, bar_end, fb_start_phys_addr, fb_end_phys_addr
6      );
7      let fb_end_aligned = PhysAddr::new(fb_end_phys_addr).align_up(PAGE_SIZE as
8          u64).as_u64();
9
10     if bar_end > fb_end_aligned {
11         let tail_start = fb_end_aligned;
12         let tail_end = bar_end;
13
14         let tail_size = tail_end - tail_start; // Für Terminalausgabe
15
16         info!("Mapping BAR{} tail at {:#x} (size: {:#x})", bar_index, tail_start,
17             tail_size);
18
19         let kernel_process = process_manager().read().kernel_process().unwrap();
20         kernel_process.virtual_address_space.kernel_map_devm_identity(
21             tail_start,
22             tail_end,
23             PageTableFlags::PRESENT | PageTableFlags::WRITABLE |
24                 PageTableFlags::NO_CACHE,
25             VmaType::DeviceMemory,
26             "virtio-gpu-vram-tail",
27         );
28     }
29     continue;
30 }
31 // Kein Overlap -> normales Mapping per address und address + size
32 ..
    
```

Die Verwendung von `NO_CACHE` ist hier eine typische Maßnahme für MMIO-/Device-Memory-Abbildungen, um unerwünschte Cache-Effekte bei Registerzugriffen zu vermeiden. Wenn kein Overlap gefunden wurde, weil die VirtIO-GPU z.B. mit `-device virtio-gpu-pci` angebunden wurde und somit nicht als VGA-Gerät fungiert, wird das BAR normal gemapped.

```

[INF][boot.rs@303] BAR0 0x80000000..0x80800000 overlaps framebuffer
0x80000000..0x801d4c00
[INF][boot.rs@315] Mapping BAR0 tail at 0x801d5000 (size: 0x62b000)
...
[INF][vmm.rs@333] (VirtAddr(0x80000000), VMA: Space: Kernel, Type: DeviceMemory,
[0x80000000; 0x801d5000], #pages: 469, tag: "framebuffer-----")
[INF][vmm.rs@333] (VirtAddr(0x801d5000), VMA: Space: Kernel, Type: DeviceMemory,
[0x801d5000; 0x80800000], #pages: 1579, tag: "virtio-gpu-vram-")

```

Anhand der Terminalausgabe kann man erkennen, dass der LFB-Framebuffer wiederverwendet wurde und der VirtIO-GPU-Framebuffer den vollen Speicher von 0x80000000 bis 0x80800000 belegt.

3.4.3 Aktivierung und Interrupt-Anbindung

Nach dem Mapping werden im Bootpfad die nötigen PCI-Command-Bits gesetzt, sodass das Gerät Bus Mastering und Memory/IO-Zugriffe nutzen darf. Anschließend wird die hardwareseitige Interrupt-Line ausgelesen. Um Konflikte mit den für CPU-Exceptions reservierten Vektoren (0–31) zu vermeiden, erfolgt eine Abbildung auf den benutzerdefinierbaren Bereich durch Addition eines Offsets von 32. Der resultierende Interrupt-Vektor wird im System registriert und dem VirtIO-Treiber zugewiesen.

```

boot.rs Rust
1 if let Some(mut device_writer) = device_lock.try_write() {
2     device_writer.update_command(pci_bus.config_space(), |cmd| {
3         cmd | pci_types::CommandRegister::BUS_MASTER_ENABLE
4             | pci_types::CommandRegister::MEMORY_ENABLE
5             | pci_types::CommandRegister::IO_ENABLE
6     });
7
8     let (_, interrupt_line) = device_writer.interrupt(pci_bus.config_space());
9     if interrupt_line != 0 && interrupt_line != 0xFF {
10         let interrupt_vector = InterruptVector::try_from(interrupt_line +
11             32).unwrap();
12         interrupt_dispatcher().assign(interrupt_vector,
13             Box::new(VirtioInterruptHandler));
14         apic().allow(interrupt_vector);
15     }
16 }

```

Damit ist die Grundlage geschaffen, Geräteereignisse später zu verarbeiten. Die eigentliche Logik der Interruptbehandlung wird im separaten Abschnitt zum VirtIO-Interrupt-Handler erläutert.

3.4.4 Treiberinstanzen und zentrale Bereitstellung

Im letzten Schritt wird aus der `PCI-Device_function` ein `PciTransport` erzeugt und anhand des Gerätetyps der passende Treiber instanziiert. Die Treiberinstanzen werden über ein einmaliges Initialisierungsmuster `call_once` erzeugt und anschließend zentral gehalten, typischerweise in Kombination mit einem `Mutex` zur Synchronisation.

```

boot.rs Rust
1 match PciTransport::new::<HalImpl, _>(&mut pci_root, device_function) {
2     Ok(mut transport) => {
3         match transport.device_type() {
4             virtio::transport::DeviceType::GPU => {
5                 VIRTIO_GPU.call_once(|| {
6                     Mutex::new(
7                         VirtIOGpu::<HalImpl, PciTransport>::new(transport)
8                             .expect("Failed to create VirtIO GPU driver")
9                     )
10                });
11            }
12            virtio::transport::DeviceType::Sound => {
13                VIRTIO_SOUND.call_once(|| {
14                    Mutex::new(
15                        VirtIOSound::<HalImpl, PciTransport>::new(transport)
16                            .expect("Failed to create VirtIO Sound driver")
17                    )
18                });
19            }
20            // weitere Gerätetypen analog
21            dt => {
22                warn!("Unbehandelter Typ: {:?}", dt);
23            }
24        }
25    }
26    Err(e) => error!("Fehler: {:?}", e),
27 }

```

Damit ist der Bootpfad so aufgebaut, dass VirtIO-Geräte erkannt, ihre Ressourcen gemappt, Interrupts angebunden und Treiberinstanzen reproduzierbar bereitgestellt werden. Auf dieser Grundlage können anschließend die gerätespezifischen Funktionalitäten (VirtIO-GPU inkl. Resize/VirGL und VirtIO-Sound) genutzt und in der Evaluation verglichen werden.

3.5 Interrupt- und Event-Anbindung

Dieser Kapitelabschnitt basiert auf Änderungen an `\os\kernel\src\interrupt\virtio_handler.rs`, da dort die zentrale Behandlung von VirtIO-bezogenen Interrupts implementiert ist. Ziel dieses Handlers ist es, Geräteinterrupts zuverlässig zu quittieren und Ereignisse so zu signalisieren, dass die eigentliche Verarbeitung außerhalb des Interrupt-Kontexts erfolgen kann. Dadurch bleibt der Interruptpfad kurz und deterministisch, während komplexere Logik in regulären Ausführungskontexten abgearbeitet wird.

Die Implementierung erfolgt über eine Struktur `VirtioInterruptHandler`, die das systeminterne InterruptHandler-Interface implementiert. Der Einstiegspunkt ist die Methode `trigger()`. Innerhalb dieser Methode werden die jeweils initialisierten VirtIO-Geräteinstanzen abgefragt und nicht-blockierend über `try_lock()` gelockt. Das nicht-blockierende Locking reduziert das Risiko, im Interrupt-Kontext in Wartezustände zu geraten, und verhindert damit unnötige Latenz oder potenzielle Verklemmungen.

Für jedes Gerät wird zunächst der Treiber gelockt, anschließend wird der Interrupt über `ack_interrupt()` quittiert und bei Bedarf eine minimale Signalisierung ausgelöst.

```

virtio_handler.rs Rust
1 pub struct VirtioInterruptHandler;
2
3 impl InterruptHandler for VirtioInterruptHandler {
4     fn trigger(&self) {
5         // GPU Handler
6         if let Some(gpu) = virtio_gpu() {
7             if let Some(mut g) = gpu.try_lock() {
8                 let st = g.ack_interrupt();
9                 // ...
10            }
11        }
12        // Sound Handler
13        if let Some(sound_dev) = virtio_sound() {
14            if let Some(mut sound_driver) = sound_dev.try_lock() {
15                let status = sound_driver.ack_interrupt();
16            }
17        }
18        // Socket Handler - nutzt polling
19    }
20 }

```

Für die Grafikunterstützung ist nicht nur das Quittieren des Interrupts relevant, sondern auch die Unterscheidung verschiedener Ursachen. VirtIO signalisiert beispielsweise **Queue-Interrupts** (abgeschlossene Queue-Aktivität) und **Konfigurationsinterrupts** (Gerätekonfigurationsänderungen). Die Implementierung wertet hierzu den zurückgegebenen `InterruptStatus` aus und setzt atomare Pending-Flags. Diese Flags dienen als Übergabemechanismus an nachgelagerte Verarbeitungsschritte, ohne dass im Interrupt-Kontext umfangreiche Arbeit durchgeführt werden muss.

`GPU_QUEUE_PENDING` markiert dabei Queue-Ereignisse, `GPU_CONFIG_PENDING` markiert Konfigurationsänderungen, die insbesondere für Resize-Unterstützung relevant sind.

```

virtio_handler.rs Rust
1 // ... GPU Handler
2 let st = g.ack_interrupt();
3
4 if st.contains(InterruptStatus::QUEUE_INTERRUPT) {
5     GPU_QUEUE_PENDING.store(true, Ordering::Release);
6 }
7 if st.contains(InterruptStatus::DEVICE_CONFIGURATION_INTERRUPT) {
8     GPU_CONFIG_PENDING.store(true, Ordering::Release);
9 }

```

Damit wird eine klare Trennung erreicht: Der Interrupt-Handler reduziert sich auf das Quittieren des Interrupts und das Setzen minimaler Zustandsindikatoren. Die eigentliche Bearbeitung, etwa das Abholen fertiggestellter Queue-Operationen oder das Reagieren auf Konfigurationsänderungen, kann anschließend in einem passenden Ausführungskontext erfolgen.

Analog zur GPU werden auch weitere VirtIO-Geräte im Handler quittiert. Für VirtIO-Sound wird der Interrupt ebenfalls per `ack_interrupt()` bestätigt und protokolliert. Dieses Schema lässt sich generisch auf weitere Geräte übertragen, die eine hardwareseitige Quittierung erfordern. Ausgenommen hiervon sind lediglich Treiberkomponenten, die ent-

wurfsbedingt auf Interrupts verzichten. Ein Beispiel hierfür ist der VirtIO-Socket-Treiber, der stattdessen Polling verwendet. Dadurch bleibt der Handler modular erweiterbar, ohne den grundlegenden Pfad für interrupt-gesteuerte Geräte zu verändern.

3.6 Globales Zugriffskonzept

Dieser Kapitelabschnitt basiert auf Änderungen an `\os\kernel\src\lib.rs`, da dort die zentralen, global verfügbaren Treiberinstanzen sowie die zugehörigen Zugriffsfunktionen definiert sind. Die in Abschnitt 3.4 initialisierten VirtIO-Treiber sollen im weiteren Systemverlauf von unterschiedlichen Komponenten genutzt werden können, etwa von Demonstrationsprogrammen, von Interruptpfaden sowie von höherliegenden Subsystemen. Dafür wird ein globales Zugriffskonzept verwendet, das eine einmalige Initialisierung sicherstellt und gleichzeitig kontrollierten, synchronisierten Zugriff ermöglicht.

Die Treiberinstanzen werden als globale, statisch lebende Strukturen gehalten. Zur Gewährleistung einer einmaligen Initialisierung wird hierfür ein `Once`-Mechanismus eingesetzt. Jede Treiberinstanz wird zusätzlich durch ein `Mutex` geschützt, sodass auch bei konkurrierenden Zugriffen ein konsistenter Zustand gewährleistet bleibt.

lib.rs	Rust
1	<code>static CPU: Once<Cpu> = Once::new();</code>
2	
3	<code>static VIRTIO_RNG: Once<Mutex<VirtIORng<HalImpl, PciTransport>>> =</code>
	<code>Once::new();</code>
4	<code>static VIRTIO_GPU: Once<Mutex<VirtIOGpu<HalImpl, PciTransport>>> =</code>
	<code>Once::new();</code>
5	<code>static VIRTIO_SOUND: Once<Mutex<VirtIOSound<HalImpl, PciTransport>>> =</code>
	<code>Once::new();</code>
6	
7	<code>pub static GPU_QUEUE_PENDING: AtomicBool = AtomicBool::new(false);</code>
8	<code>pub static GPU_CONFIG_PENDING: AtomicBool = AtomicBool::new(false);</code>

Die zusätzlichen atomaren Statusvariablen dienen der Ereignissignalisierung aus dem Interrupt-Kontext. Wie in Abschnitt 3.5 gezeigt, werden sie durch den Interrupt-Handler gesetzt und anschließend außerhalb des Interrupt-Kontexts ausgewertet. Damit bleibt der Interruptpfad kurz, während die eigentliche Verarbeitung im regulären Ausführungskontext erfolgt.

Um den Zugriff auf die Treiberinstanzen zu vereinheitlichen, werden Getter-Funktionen bereitgestellt. Diese kapseln die globale Ablage und liefern bei erfolgreicher Initialisierung eine Referenz auf das jeweilige `Mutex`-geschützte Treiberobjekt.

Der Rückgabewert ist jeweils Option, da eine Treiberinstanz nur dann verfügbar ist, wenn sie im Boot-Prozess tatsächlich initialisiert wurde. Dadurch bleibt das System robust gegenüber Konfigurationen, in denen bestimmte VirtIO-Geräte nicht vorhanden sind.

```

lib.rs Rust
1 pub fn virtio_gpu() -> Option<&'static Mutex<VirtIOGpu<HalImpl, PciTransport>>> {
2     VIRTIO_GPU.get()
3 }
4 pub fn virtio_sound() -> Option<&'static Mutex<VirtIOSound<HalImpl,
5     PciTransport>>> {
6     VIRTIO_SOUND.get()
7 }
8 // Möglichkeit für weitere Devices

```

Dieses Muster erfüllt zwei Ziele, erstens wird die Initialisierung semantisch klar vom Zugriff getrennt. Zweitens wird der Umgang mit optional vorhandenen Geräten zentral geregelt, sodass aufrufende Komponenten keinen direkten Zugriff auf globale Variablen benötigen und die Abhängigkeiten transparent bleiben.

Das Singleton- und Zugriffskonzept bildet den verbindenden Baustein zwischen den zuvor beschriebenen Teilen. Die Initialisierung in `boot.rs` erzeugt die Treiberinstanzen einmalig und legt sie in den globalen `Once`-Containern ab. Der Interrupt-Handler nutzt die Getter-Funktionen, um vorhandene Geräte nicht-blockierend zu locken, und Interrupts zu quittieren. Höherliegende Komponenten, wie die Grafik- und Audio-Demos, greifen über dieselben Getter-Funktionen auf die Treiber zu und profitieren dadurch von einem einheitlichen Zugriffspfad.

3.7 VirtIO-Sound und Beispielanwendung

Neben der Grafikerunterstützung wurde in dieser Arbeit auch VirtIO-Sound angebunden, um eine einfache Audioausgabe in D3OS zu ermöglichen. Ziel ist dabei nicht die vollständige Abdeckung aller Features der VirtIO-Sound-Spezifikation, sondern eine minimal funktionsfähige PCM-Playback-Pipeline, die als Nachweis dient, dass die integrierte Treiberbasis auch zeitkontinuierliche Datenströme (Audio) zuverlässig bedienen kann.

Treiberaufbau und Queues

Der VirtIO-Sound-Treiber folgt, analog zu anderen VirtIO-Geräten, dem üblichen Modell aus Konfigurationsraum, Feature-Negotiation und mehreren Virtqueues. Beim Initialisieren werden aus dem Konfigurationsraum die verfügbaren Jacks, Streams und Channel Maps gelesen. Anschließend werden vier Queues aufgebaut:

- Control-Queue: Austausch von Kontrollnachrichten (z. B. Abfrage von Stream-Informationen, Setzen von Parametern, Start/Stop).
- Event-Queue: Ereignisse/Benachrichtigungen des Geräts (z. B. Period-Notifications).
- TX-Queue: Übertragung von PCM-Daten vom Gast zum Host (Playback).
- RX-Queue: Empfangspfad (z. B. Capture); in dieser Arbeit nicht weiter genutzt.

Der Treiber kapselt diese Queues in einer Instanz und stellt darauf aufbauend eine kompakte PCM-API bereit. Diese umfasst Funktionen wie `pcm_set_params`, `pcm_prepare`, `pcm_start`, `pcm_xfer`, `pcm_stop` und `pcm_release`. Zur Fähigkeitserkennung werden

zusätzlich die pro Stream unterstützten Raten, Formate und Kanalbereiche aus den PCM-Infos extrahiert.

PCM-Parameter

Für den PCM-Betrieb sind insbesondere zwei Parameter entscheidend:

- `period_bytes`: Größe eines einzelnen Audioblocks (eine „Periode“), der in einem Transfer übergeben wird.
- `buffer_bytes`: Größe des Ringpuffers im Gerät, typischerweise ein Vielfaches der Periodengröße.

Der Treiber validiert dabei, dass `period_bytes > 0`, `period_bytes <= buffer_bytes` und `buffer_bytes % period_bytes == 0`. Diese Constraints spiegeln die Erwartung wider, dass das Gerät periodisch konsumiert und die Transfers in periodengroße Blöcke zerlegt werden. In der Testanwendung wird außerdem darauf geachtet, dass `period_bytes` frame-aligned ist, also durch die Framegröße (Bytes pro Sample × Kanalanzahl) teilbar bleibt. Dadurch wird verhindert, dass Transfers mitten in einem Sample-Frame enden.

Blocking vs. Non-Blocking Transfer

Für das Senden von Audiodaten existieren zwei Varianten:

- `pcm_xfer` (blocking): zerlegt den Eingabepuffer in Perioden und hält die TX-Queue so lange gefüllt, bis alle Blöcke abgespielt wurden. Die Verarbeitung der Completions erfolgt über `pop_used`, sodass der Aufrufer blockiert, bis die Wiedergabe abgeschlossen ist.
- `pcm_xfer_nb` (non-blocking): übergibt genau eine Periode und gibt sofort ein Token zurück. Der Treiber behält die zugehörigen Buffer/Statusstrukturen intern, bis der Aufrufer später mit `pcm_xfer_ok` (token) die Completion quittiert. Dieses Modell erlaubt es, mehrere Perioden „in flight“ zu halten, und damit eine kontinuierliche Wiedergabe ohne lange kritische Abschnitte zu realisieren.

Gerade in einem Bare-Metal-Kontext ist der non-blocking Pfad nützlich, weil er eine feinere Kontrolle über Scheduling und Synchronisation ermöglicht: Statt eine große Transferfunktion zu blockieren, kann die Anwendung periodisch Tokens einsenden und completete Transfers nachziehen.

Beispielanwendung

Zur Demonstration wurde die Funktion `play_pcm_file()`, welche über die `boot.rs` aufgerufen wird und sich in `os\kernel\src\virtio_demo\virtio_tests` befindet, implementiert. Sie stellt einen minimalen Audioplayer dar, der eine PCM-Datei zur Kompilierzeit über das Makro (`include_bytes!(...)`) einbettet und diese anschließend über VirtIO-Sound ausgibt. Der Ablauf ist in mehrere Phasen gegliedert:

1. Abfrage der Gerätefähigkeiten: Zunächst wird ein Output-Stream bestimmt und die unterstützten Sample-Raten, Formate und Kanalbereiche gelesen. Daraus werden Zielparameter abgeleitet (mit einfachen Fallbacks, falls z. B. 44.1 kHz oder S16 nicht verfügbar sind).
2. Konfiguration des Streams: Mit `pcm_set_params` werden Buffer- und Periodengröße sowie Kanalzahl, Format und Rate gesetzt. Darauf folgen `pcm_prepare` und `pcm_start`.
3. Streaming der PCM-Daten: Die PCM-Daten werden in periodengroße Blöcke geschnitten und mittels `pcm_xfer_nb` eingereiht. Um eine stabile Wiedergabe zu erreichen, hält die Implementierung ein kleines „In-Flight-Fenster“ (mehrere Perioden gleichzeitig in der Queue). Parallel dazu werden
 - Benachrichtigungen aus der Event-Queue abgeholt (sofern vorhanden) und
 - fertige Transfers über `pcm_xfer_ok(token)` aus dem Used-Ring entfernt.
4. Aufräumen: Nach Abschluss werden `pcm_stop` und `pcm_release` aufgerufen

Ein weiterer zentraler Implementierungsaspekt ist die Locking-Strategie. Analog zur in Abschnitt 4.4 beschriebenen Problematik bei der Grafikdarstellung erfordert auch der Audio-Treiber Zugriffsschutz durch Mutex-Sperren. Deshalb minimiert `play_pcm_file()` die Lockdauer bewusst. Geräteabfragen und Kontrolloperationen erfolgen in kurzen Lock-Abschnitten, während die eigentliche Parameterauswahl und Chunk-Aufteilung ohne Lock stattfinden. Dadurch wird vermieden, dass längere Audiooperationen das System unnötig blockieren.

Kapitel 4

VirtIO-GPU und Erweiterungen

4.1 Erweiterung um VirGL und 3D-Kommandos

Dieser Kapitelabschnitt basiert auf Änderungen an `gpu.rs` innerhalb der lokalen virtio-drivers-Quellen, da die VirGL-Unterstützung zusätzliche Protokollbestandteile, Kommandostrukturen und Hilfsfunktionen erfordert. Ziel ist dabei nicht der Aufbau eines vollständigen 3D-Stacks, sondern die Ergänzung genau der Funktionen, die zur Ausführung der in Abschnitt 4.2 beschriebenen `test_virgl`-Routine notwendig sind.

Die Erweiterung um VirGL basiert auf der VirtIO-GPU-Spezifikation, in der für 3D-fähige Geräte zusätzliche ControlQ-Kommandos definiert sind. Damit der Treiber überhaupt 3D-Kommandos an das Gerät senden darf, wird VirGL als unterstütztes Feature in die Feature-Maske aufgenommen. Dies stellt sicher, dass die Feature-Negotiation VirGL aushandeln kann, sofern das Gerät diese Fähigkeit anbietet.

```
gpu.rs Rust  
1 const SUPPORTED_FEATURES: Features = Features::RING_EVENT_IDX  
2   .union(Features::RING_INDIRECT_DESC)  
3   .union(Features::VERSION_1)  
4   .union(Features::VIRGL); // neu
```

Die clientseitige Aktivierung des Feature-Bits ist jedoch nur wirksam, wenn die Virtualisierungsumgebung die entsprechende 3D-Funktionalität ebenfalls exponiert. In der verwendeten QEMU-Konfiguration (definiert in `Makefile.toml`) muss hierfür das Gerät `virtio-vga-gl` sowie ein OpenGL-fähiges Display-Backend aktiviert werden. Dies erfolgt über die Startparameter:

```
Makefile.toml TOML  
1 # VirtIO GPU  
2 "-display", "gtk,gl=on", "-device", "virtio-vga-gl"
```

Erst durch diese Parameter bietet das Gerät das VIRGL-Featurebit beim Handshake an. Ob die Aushandlung erfolgreich war, lässt sich zur Laufzeit anhand der Debug-Ausgaben der Treiberinitialisierung verifizieren. Eine Ausgabe wie:

```
...  
[0.266][INF][gpu.rs@366] [gpu device] negotiated_features: Features(VIRGL |  
RING_INDIRECT_DESC | RING_EVENT_IDX | VERSION_1)
```

dokumentiert die Schnittmenge der vom Host angebotenen und vom Gast akzeptierten Features. Enthält diese Liste das VIRGL-Flag nicht, stehen die 3D-Kommandos (ab 0x200) nicht zur Verfügung.

Damit der Treiber diese nun ausgehandelte Funktionalität nutzen kann, müssen die entsprechenden Protokollwerte in der Software abgebildet werden. Zunächst werden die 3D-spezifischen Command-IDs ergänzt. Sie liegen im Bereich 0x200 bis 0x207 und decken Kontextverwaltung, 3D-Resource-Erzeugung, Transfers und Command-Submission ab.

gpu.rs	Rust
1	<code>const CTX_CREATE: Command = Command(0x200);</code>
2	<code>const CTX_DESTROY: Command = Command(0x201);</code>
3	<code>const CTX_ATTACH_RESOURCE: Command = Command(0x202);</code>
4	<code>const CTX_DETACH_RESOURCE: Command = Command(0x203);</code>
5	<code>const RESOURCE_CREATE_3D: Command = Command(0x204);</code>
6	<code>const TRANSFER_TO_HOST_3D: Command = Command(0x205);</code>
7	<code>const TRANSFER_FROM_HOST_3D: Command = Command(0x206);</code>
8	<code>const CMD_SUBMIT_3D: Command = Command(0x207);</code>

Die zugehörigen Request-Strukturen werden im Treiber als `#[repr(C)]`-Strukturen definiert. Damit ist sichergestellt, dass das Speicherlayout den Spezifikationsstrukturen entspricht. In Kombination mit den verwendeten Zerocopy-Traits (`IntoBytes`, `FromBytes`, `KnownLayout`, `Immutable`) können Requests deterministisch in einen Sendepuffer geschrieben und Responses zuverlässig aus einem Empfangspuffer gelesen werden.

Die neu eingeführten Typen werden im Folgenden funktional gruppiert:

- Kontextverwaltung: `CtxCreate`, `CtxDestroy`
Erzeugt bzw. zerstört einen 3D-Kontext, an den Ressourcen gebunden werden können.
- Kontext-Ressource-Bindung: `CtxAttachResource`, `CtxDetachResource`
Verknüpft eine 3D-Resource mit einem Kontext (Voraussetzung für Submission).
- 3D-Resource: `ResourceCreate3D`
Erzeugt eine 3D-Resource mit Parametern wie Format, Bind-Flags und Abmessungen.
- 3D-Transfer: `TransferToHost3D`, `TransferFromHost3D` und Hilfstyp `Box3D`
transferiert eine 3D-Region-Box zwischen Guest-Backing und Host.
- Submit: `Submit3D`
Trägt Metadaten zur Länge des nachfolgenden Command-Streams; die Bytes selbst werden separat übertragen.

Der folgende Quelltext zeigt exemplarisch eine der zentralen Nachrichten, `ResourceCreate3D`, die im Smoke Test als Render-Target angelegt wird:

```

gpu.rs Rust
1  #[repr(C)]
2  #[derive(Debug, Immutable, IntoBytes, KnownLayout)]
3  struct ResourceCreate3D {
4      header: CtrlHeader,
5      resource_id: u32,
6      target: u32,
7      format: u32,
8      bind: u32,
9      width: u32,
10     height: u32,
11     depth: u32,
12     array_size: u32,
13     last_level: u32,
14     nr_samples: u32,
15     flags: u32,
16     _padding: u32,
17 }

```

Ein wesentlicher Unterschied zur 2D-Implementierung ist, dass bestimmte 3D-Kommandos neben dem festen Request-Header einen zusätzlichen Datenpuffer übertragen müssen:

- bei `CTX_CREATE` wird der Debug-Name zusätzlich als Bytefolge übertragen,
- bei `CMD_SUBMIT_3D` folgt der eigentliche Command-Stream als Byte-Slice.

Dafür wird eine zusätzliche Request-Funktion eingeführt, die Header und Daten als verkettete Descriptor-Kette an die Control-Queue übergibt. Dadurch bleibt die Nachrichtenstruktur typisiert, während variable Daten ohne manuelle Kopierlogik mitgesendet werden können.

```

gpu.rs Rust
1  fn request_with_data<Req: IntoBytes + Immutable, Rsp: FromBytes>(
2      &mut self,
3      req: Req,
4      data: &[u8],
5  ) -> Result<Rsp> {
6      req.write_to_prefix(&mut self.queue_buf_send).unwrap();
7      let header_len = core::mem::size_of::<Req>();
8
9      self.control_queue.add_notify_wait_pop(
10         &[&self.queue_buf_send[..header_len], data],
11         &mut [&mut self.queue_buf_recv],
12         &mut self.transport,
13     )?;
14     Ok(Rsp::read_from_prefix(&self.queue_buf_recv).unwrap().0)
15 }

```

Auf Basis der neuen Strukturen werden Treibermethoden ergänzt, die jeweils eine konkrete Protokolloperation kapseln und im Erfolgsfall `OK_NODATA` erwarten. Beispiele sind:

- `resource_create_3d(...)` → sendet `RESOURCE_CREATE_3D`
- `context_create(ctx_id, name)` → sendet `CTX_CREATE` + Name über `request_with_data`

- `ctx_attach_resource(ctx_id, res_id)` → sendet `CTX_ATTACH_RESOURCE`
- `transfer_from_host_3d(...)` → sendet `TRANSFER_FROM_HOST_3D`
- `submit_3d(ctx_id, commands)` → sendet `CMD_SUBMIT_3D` + Command-Stream über `request_with_data`

Der nachfolgende Quelltext zeigt die Submission-Funktion als repräsentatives Beispiel. Hier wird zusätzlich eine Fence-ID gesetzt, um Submissions zur Synchronisation mit Sequenznummern eindeutig zu markieren.

```

gpu.rs Rust
1 fn submit_3d(&mut self, ctx_id: u32, commands: &[u8]) -> Result {
2     let mut header = CtrlHeader::with_type(Command::CMD_SUBMIT_3D);
3     header.ctx_id = ctx_id;
4
5     static mut NEXT_FENCE: u64 = 1;
6     let fence = unsafe { let f = NEXT_FENCE; NEXT_FENCE += 1; f };
7     header.flags = GPU_FLAG_FENCE;
8     header.fence_id = fence;
9
10    let submit_cmd = CmdSubmit3D { header, size: commands.len() as u32, _padding:
11        0 };
12
13    let rsp: CtrlHeader = self.request_with_data(submit_cmd, commands)?;
14    rsp.check_type(Command::OK_NODATA)
15 }
```

4.2 VirGL Smoke Test

Dieser Kapitelabschnitt basiert auf der in `gpu.rs` implementierten Funktion `test_virgl` sowie den hierfür ergänzten Encoder-Routinen zur Erzeugung eines VirGL-Command-Streams. Ziel des Smoke Tests ist es, den vollständigen 3D-Pfad einmal „end-to-end“ zu durchlaufen. Ein minimaler 3D-Render-Schritt wird am Host ausgelöst. Anschließend wird das Ergebnis in den Gast zurückgelesen und über den bestehenden 2D-Framebuffer sichtbar gemacht. Damit wird die prinzipielle Nutzbarkeit der VirGL-Erweiterung nachgewiesen, ohne einen vollständigen 3D-Stack im Gast vorauszusetzen.

Die im Smoke Test verwendeten Encoder sind an die Definitionen aus `virgl_protocol.h` angelehnt, wie sie z. B. in `virglrenderer` bereitgestellt werden [15]. Ein VirGL-Kommando besteht aus einem 32-Bit Headerwort mit folgendem Layout: 8 Bit Command, 8 Bit Objekt-Typ und 16 Bit Länge (in 32-Bit-Wörtern). Dieses Layout wird durch das Makro `VIRGL_CMD0(cmd, obj, len)` beschrieben.

Der Folgende Code zeigt die dazu passende Hilfsfunktion `virgl_cmd0` sowie die elementaren „push“-Funktionen zur Little-Endian-Serialisierung. Die Command-Streams werden als Byte-Vektor aufgebaut, wobei die Protokolleinheiten jeweils als Folge von 32-Bit-Wörtern (und bei Bedarf 64-Bit-Floats) kodiert werden.

```

gpu.rs Rust
1 #[inline]
2 fn virgl_cmd0(cmd: u32, obj: u32, len_u32: u32) -> u32 {
3     // entspricht VIRGL_CMD0(cmd,obj,len)
4     cmd | (obj << 8) | (len_u32 << 16)
5 }
6
7 #[inline]
8 fn push_u32(buf: &mut Vec<u8>, v: u32) { buf.extend_from_slice(&v.to_le_bytes());
9     }
10
11 #[inline]
12 fn push_f32(buf: &mut Vec<u8>, f: f32) { push_u32(buf, f.to_bits()); }
13
14 #[inline]
15 fn push_f64(buf: &mut Vec<u8>, d: f64) {
16     let bits = d.to_bits();
17     buf.extend_from_slice(&(bits as u32).to_le_bytes()); // low 32
18     buf.extend_from_slice(&((bits >> 32) as u32).to_le_bytes()); // high 32
19 }

```

Auf Basis dieser Hilfsfunktionen wird ein kleiner Satz an Encodern implementiert, der für den Smoke Test ausreicht. Inhaltlich bilden die Encoder jeweils genau eine Protokolloperation aus `virgl_protocol.h` ab: Surface erzeugen, Framebuffer-State setzen, Viewport setzen und einen Clear durchführen.

Im Folgenden wird exemplarisch der Encoder `encode_create_surface` gezeigt. Jeder Encoder beginnt mit dem Schreiben eines 32-Bit-Headerworts, das über `virgl_cmd0` erzeugt wird. Der Parameter `len` gibt dabei die Anzahl der unmittelbar folgenden Payload-Wörter in 32 Bit an. Entsprechend werden nach dem Header genau `len` Werte per `push_u32` in den Command-Stream geschrieben. Im Beispiel ist `len = 5`, weshalb auf den Header genau fünf 32-Bit-Wörter folgen.

```

gpu.rs Rust
1 fn encode_create_surface(buf: &mut Vec<u8>, surf_handle: u32, res_handle: u32,
2     format: u32) {
3     // Header: Command + Objekt + Payload-Länge (in u32-Wörtern)
4     push_u32(buf, virgl_cmd0(VIRGL_CCMD_CREATE_OBJECT, VIRGL_OBJECT_SURFACE, 5));
5
6     // Payload: genau 5 u32-Wörter gemäß len=5
7     push_u32(buf, surf_handle);
8     push_u32(buf, res_handle);
9     push_u32(buf, format);
10    push_u32(buf, 0); // level = 0
11    push_u32(buf, 0); // layers = 0 (first=0,last=0)
12 }

```

Die weiteren im Smoke Test genutzten Encoder folgen demselben Schema: Zunächst wird ein Headerwort mit der passenden Command-ID und Payload-Länge geschrieben, anschließend werden die jeweils spezifizierten Parameter in der vorgegebenen Reihenfolge serialisiert. Dadurch entsteht ein deterministischer Byte-Stream, der anschließend über `CMD_SUBMIT_3D` an den Host übertragen werden kann.

Der Smoke Test setzt die in Abschnitt 4.1 beschriebenen 3D-Erweiterungen in einer festen Abfolge ein. Diese Abfolge ist so gewählt, dass sie mit minimalem Umfang eine vollständige

3D-Roundtrip-Interaktion abbildet:

1. `context_create`: Erzeugt einen 3D-Kontext, der als Ausführungskontext für Submissions dient.
2. `resource_create_3d`: Erzeugt eine 3D-Resource als Render-Target in der gewünschten Auflösung.
3. `ctx_attach_resource`: Bindet die Resource an den Kontext.
4. `encode_*`: Erzeugt einen minimalen Virgl-Command-Stream (Surface, FB-State, Viewport, Clear).
5. `submit_3d`: Sendet den Command-Stream an den Host. Optional wird eine Fence-ID gesetzt, um Submissions auf einer Synchronisations-Timeline eindeutig zu markieren.
6. `flush`: Bild sichtbar machen.

Der folgende Code zeigt diese Pipeline als strukturierte Sequenz (schematisch, d. h. ohne alle Parameter):

```
gpu.rs Rust
1 // 1) Kontext + 3D-Resource
2 context_create(ctx_id, "smoke"?;
3 resource_create_3d(res_id, /*target, format, bind, w, h, ...*/)?;
4 ctx_attach_resource(ctx_id, res_id)?;
5
6 // 2) VirGL Command Stream aufbauen
7 let mut cmds = Vec::new();
8 encode_create_surface(&mut cmds, surf_id, res_id, format);
9 encode_set_framebuffer_state(&mut cmds, surf_id, 0);
10 encode_set_viewport_full(&mut cmds, width, height);
11 encode_clear_color(&mut cmds, 0.094, 0.416, 0.067, 1.0); //RGBA Colors
12
13 // 3) Submit
14 submit_3d(ctx_id, &cmds)?;
15
16 // 4) Flush
17 resource_flush(rect, res_id)?;
```

Dieser Programmablauf demonstriert zudem den architektonischen Vorteil der 3D-Beschleunigung gegenüber dem klassischen 2D-Modus. Während bei der 2D-Implementierung zwingend ein physischer Speicherbereich (Backing Store) im Gast-RAM reserviert und dessen Inhalt mittels `TRANSFER_TO_HOST_2D` zyklisch an den Host kopiert werden muss, entfällt dieser Schritt bei der direkten 3D-Nutzung. Da der virglrenderer die Befehle des Command-Streams unmittelbar auf der Host-GPU ausführt, entstehen die Bilddaten direkt im Videospeicher der Host-Ressource. Es ist daher im Gastsystem weder das Vorhalten eines gefüllten Pixelpuffers noch ein bandbreitenintensiver Datentransfer erforderlich. Dieser Zero-Copy-Ansatz reduziert die Latenz signifikant, da zur Aktualisierung der Anzeige lediglich der `RESOURCE_FLUSH`-Befehl gesendet werden muss, der dem Hypervisor die Fertigstellung des Frames signalisiert.

Weiter wird im Smoke Test die Hintergrundfarbe über `encode_clear_color` gesetzt. Die Funktion erwartet die Farbkanäle als Gleitkommazahl im Bereich $[0,1]$, also in einer normalisierten Darstellung. Zur Einordnung kann diese Darstellung in 8-Bit-RGBA-Werte $[0,255]$ umgerechnet werden, indem mit 255 multipliziert wird. Für die im Test verwendeten Werte ergibt sich damit näherungsweise: $R = 0.094 \cdot 255 \approx 24$, $G = 0.416 \cdot 255 \approx 106$, $B = 0.067 \cdot 255 \approx 17$, $A = 1.0 \cdot 255 = 255$. Damit entspricht der Clear-Vorgang einer dunklen Grünfärbung mit voller Deckkraft.

Der Nachweischarakter des Smoke Tests ergibt sich daraus, dass eine rein 2D-basierte Implementierung die obigen Schritte 4 und 5 nicht ausführen kann. Erst durch `SUBMIT_3D` und dem `flush()` wird sichtbar, dass der Host den 3D-Command-Stream akzeptiert und verarbeitet.

4.3 Resize-Unterstützung

Die Resize-Unterstützung erfordert, dass der bestehende Framebuffer-Zustand beim Wechsel der Auflösung konsistent abgebaut und anschließend mit den neuen Parametern neu aufgebaut wird. Ursache ist, dass die Framebuffer-Ressource (`RESOURCE_ID_FB`) im Treiber mit einer festen Resource-ID (`0xbabe`) erzeugt wird, während sich bei einer Auflösungsänderung sowohl Größe als auch Backing-Speicher ändern. Ein erneuter Aufbau ohne vorherigen Abbau kann dazu führen, dass der Host eine erneute Erstellung einer bereits existierenden Resource-ID ablehnt.

Die verwendete virtio-drivers-Basis stellt die relevanten Kommando-IDs für das Entfernen des Backings und das Freigeben einer Resource bereits bereit mit:

- `RESOURCE_DETACH_BACKING` als Kommando `0x107`
- `RESOURCE_UNREF` als Kommando `0x102`

Damit war für die Resize-Funktionalität nicht notwendig, diese Kommandos neu zu definieren. Für die konkrete Nutzung im Request-Pfad mussten jedoch die zugehörigen Request-Strukturen ergänzt werden, um die Kommandos als vollständige Nachrichten an die Control-Queue senden zu können.

```

gpu.rs Rust
1  #[repr(C)]
2  #[derive(Debug, Immutable, IntoBytes, KnownLayout)]
3  struct ResourceDetachBacking {
4      header: CtrlHeader,
5      resource_id: u32,
6      _padding: u32,
7  }
8  #[repr(C)]
9  #[derive(Debug, Immutable, IntoBytes, KnownLayout)]
10 struct ResourceUnref {
11     header: CtrlHeader,
12     resource_id: u32,
13     _padding: u32,
14 }

```

Damit ist die technische Voraussetzung erfüllt, um im Resize-Fall den bisherigen 2D-Framebuffer korrekt zu lösen, bevor eine neue Resource erzeugt wird.

Hierzu wird in `setup_framebuffer()` zunächst geprüft, ob bereits ein Framebuffer existiert. Falls ja, wird der bestehende Zustand über `destroy_framebuffer()` abgebaut. In diesem Zuge wird der Scanout gelöst, das Resource-Backing getrennt und die Resource referenzseitig freigegeben. Danach wird der interne Treiberzustand so aktualisiert, dass der zugehörige DMA-Puffer freigegeben werden kann.

```

gpu.rs Rust
1 fn destroy_framebuffer(&mut self, rect_for_scanout: Rect) {
2     let _ = self.set_scanout(rect_for_scanout, SCANOUT_ID, 0);
3     let _ = self.resource_detach_backing(RESOURCE_ID_FB);
4     let _ = self.resource_unref(RESOURCE_ID_FB);
5
6     self.frame_buffer_dma = None;
7     self.rect = None;
8 }

```

Ohne die beschriebene Cleanup-Routine tritt in der Konfiguration mit Ubuntu 25.10 als Host/Stack bei einem Resize reproduzierbar ein Host-seitiger Fehler auf, der sich in der Ausgabe als:

```

virtio_gpu_virgl_process_cmd: ctrl 0x101, error 0x1203

```

manifestiert. Aus Sicht der VirtIO-GPU-Kommandos deutet dies darauf hin, dass beim erneuten Framebuffer-Aufbau ein `RESOURCE_CREATE_2D` für eine Resource-ID abgesetzt wird, die aus Host-Sicht nicht in einem gültigen Zustand ist oder bereits existiert. Die Zuordnung von Kommando- und Fehlercode ist dabei konsistent mit den in Referenzimplementierungen dokumentierten Konstanten (`0x101` für `RESOURCE_CREATE_2D`, `0x1203` für `ERR_INVALID_RESOURCE_ID`).

Interessant ist, dass dieses Verhalten hostabhängig variiert. In einer WSL2-basierten Umgebung wurde beobachtet, dass der Resize auch ohne explizites `destroy_framebuffer` funktioniert. Eine mögliche Erklärung ist, dass sich die beteiligten Host-Komponenten (QEMU-Version, virglrenderer, Grafik-Backend und deren Fehlerbehandlung) unterscheiden und dadurch unterschiedliche Toleranzen gegenüber erneut verwendeten Resource-IDs entstehen.

4.4 Demos: Kompatibilität und Synchronisation

Dieser Kapitelabschnitt basiert auf Änderungen an `\os\kernel\src\virtio_demo\rectangle_demo.rs`, die Änderungen sind auch analog übertragbar auf die `pong_demo.rs`. Hintergrund der Änderungen ist insbesondere, dass der integrierte Treiber als generische Struktur `VirtIOGpu<H, T>` implementiert ist, wobei H für die HAL und T für den Transport steht. Treiberfunktionen wie `setup_framebuffer` und `flush()` sind dabei als Methoden mit `&mut self` definiert, beispielsweise:

```

gpu.rs Rust
1 pub fn setup_framebuffer(&mut self) -> Result<&mut [u8]> { ... }

```

Aus dieser Methodensignatur folgt, dass parallele oder verschachtelte Zugriffe auf dieselbe

Treiberinstanz kontrolliert werden müssen. In der Vorversion der Demo wurde ein direkter GPU-Zugriff über eine eigene, nicht generische Treiberabstraktion verwendet und es war kein Mutex-basiertes Zugriffskonzept erforderlich. Die neue Demo übergibt stattdessen eine Mutex-geschützte Treiberinstanz und führt GPU-Operationen in einem kritischen Abschnitt aus, um die erforderliche exklusive Mutabilität herzustellen.

```
rectangle_demo.rs Rust
1 pub fn rectangle_demo(gpu_mutex: &Mutex<VirtIOGpu<HalImpl, PciTransport>>) {
2     let (mut fb_ptr, mut fb_len, mut screen_width, mut screen_height) =
3         interrupts::without_interrupts(|| {
4             let mut gpu = gpu_mutex.lock();
5             let (w, h) = gpu.resolution().unwrap();
6             let buf = gpu.setup_framebuffer().unwrap();
7             (buf.as_mut_ptr(), buf.len(), w as usize, h as usize)
8         });
9     // ...
10 }
```

Da dieselbe GPU-Instanz auch im Interruptpfad verwendet werden kann, ist es hier essenziell den kritischen Abschnitt durch `interrupts::without_interrupts` zu schützen. Würde der Thread den Mutex halten und von einem GPU-Interrupt unterbrochen werden, schläge der Versuch des Handlers, den Mutex mittels `try_lock` zu akquirieren, fehl. Infolgedessen könnte der Handler den Interrupt hardwareseitig nicht bestätigen (Acknowledge). Nach Rückkehr aus dem Handler würde die CPU aufgrund des weiterhin anliegenden Signals den Interrupt sofort erneut auslösen. Dies führt zu einem „Interrupt Storm“, der eine starke Verlangsamung des Systems verursacht.

Dabei ist entscheidend, dass diese kritischen Abschnitte bewusst so kurz wie möglich gehalten werden. Ziel ist, den `Mutex` schnell wieder freizugeben und Interrupts unmittelbar danach wieder zuzulassen, um Latenz und Nebenwirkungen auf andere Subsysteme zu minimieren. In der Praxis wird daher nur der minimale Treiberaufruf innerhalb des kritischen Abschnitts ausgeführt, während rechenintensive Arbeit wie Rendering-Schleifen, Pixelberechnungen oder Buffer-Manipulation außerhalb erfolgt.

```
rectangle_demo.rs Rust
1 interrupts::without_interrupts(|| {
2     let mut gpu = gpu_mutex.lock();
3     gpu.flush().expect("GPU flush failed");
4 });
```

Dieses Vorgehen hat jedoch einen messbaren Nebeneffekt. Während `interrupts::without_interrupts` aktiv ist, sind auch Timer-Interrupts ausgesetzt. Da die Demo zur frames-per-second (FPS)-Berechnung `sys_get_system_time()` verwendet und diese Zeitbasis direkt von periodischen Timer-Interrupts abhängt, kann dies die Messung verfälschen oder im Extremfall zu unplausiblen Ergebnissen führen. Aus diesem Grund ist bei der Implementierung darauf zu achten, dass Zeitmessungen für FPS-Berechnungen nicht innerhalb interruptfreier Abschnitte erfolgen und dass die interruptfreien Bereiche insgesamt kurz bleiben. Für die Evaluation wird dieses Verhalten berücksichtigt.

Weiterhin wird in der aktualisierten Demo die Resize-Unterstützung implementiert. Dazu wird ein Konfigurationsereignis über ein Pending-Flag signalisiert und anschließend im

normalen Game-Loop verarbeitet. Im Resize-Fall werden Auflösung und Framebuffer-Slice neu bezogen. Da sich die Framebuffer-Adresse nach einem Resize ändern kann, erzeugt die Demo den Graphics-Wrapper pro Iteration neu auf Basis des aktuellen Pointers.

```
rectangle_demo.rs Rust  
1 if GPU_CONFIG_PENDING.swap(false, Ordering::Acquire) {  
2     interrupts::without_interrupts(|| {  
3         let mut gpu = gpu_mutex.lock();  
4         if let Ok((w, h)) = gpu.resolution() {  
5             screen_width = w as usize;  
6             screen_height = h as usize;  
7         }  
8         if let Ok(buf) = gpu.setup_framebuffer() {  
9             fb_ptr = buf.as_mut_ptr();  
10            fb_len = buf.len();  
11        }  
12    });  
13 }
```

```
rectangle_demo.rs Rust  
1 // Create Graphics wrapper for the current frame  
2 // We do this every loop iteration because 'fb_ptr' might have changed.  
3 let stride = screen_width * 4;  
4 let fb_slice = unsafe { slice::from_raw_parts_mut(fb_ptr, fb_len) };  
5 let mut gfx = Graphics::new(fb_slice, screen_width, screen_height, stride);
```

Kapitel 5

Evaluation

Dieses Kapitel bewertet die praktische Nutzbarkeit der implementierten VirtIO-Treiberintegration in drei Dimensionen: (i) funktionale Verifikation der VirGL-Erweiterung über einen Smoke Test, (ii) funktionaler Nachweis der Audioausgabe über VirtIO-Sound anhand eines PCM-Playback-Tests und (iii) quantitative Leistungsbewertung der Grafikpipeline anhand der Rectangle-Demo. Die Messlogik der Demo orientiert sich an in der Vorarbeit etablierten Vorgehen, bei dem Frames gezählt und die verstrichene Zeit über `sys_get_system_time()` bestimmt wird.

Alle Demos werden dabei aus der `boot.rs` Datei heraus mit den Funktionsaufrufen `test_virgl()`, `play_pcm_file()` und `rectangle_demo()` gestartet.

5.1 Ausgabe des VirGL Smoke Tests

Der VirGL Smoke Test `test_virgl` dient als End-to-End-Nachweis dafür, dass der 3D-Command-Stream vom Host akzeptiert und verarbeitet wird. Der Test durchläuft dabei die in Kapitel 4.2 beschriebenen Schritte (Kontext erzeugen, 3D-Resource erstellen, Command Stream submitten, Darstellung über Flush).

Zur Dokumentation werden die wichtigsten Debug-Ausgaben herangezogen, die die erfolgreiche Abarbeitung der einzelnen Etappen zeigen. Die folgende Terminalausgabe enthält einen gekürzten Log-Auszug.

```
[0.274] [INF] [gpu.rs@904] Starting Virgl 3D test...
virtio_gpu_cmd_ctx_create ctx 0x1, name test_ctx
virtio_gpu_cmd_get_display_info
virtio_gpu_cmd_res_create_3d res 0x2000, fmt 0x1, w 640, h 480, d 1
virtio_gpu_cmd_ctx_res_attach ctx 0x1, res 0x2000
virtio_gpu_cmd_set_scanout id 0, res 0x2000, w 640, h 480, x 0, y 0
...
Sende 108 Bytes an 3D-Befehlen...
virtio_gpu_cmd_ctx_submit ctx 0x1, size 108
virtio_gpu_fence_ctrl fence 0x1, type 0x207
virtio_gpu_fence_resp fence 0x1
ctx_submit rsp type=0x1100, fence_id=0x1, ctx=0x1
virtio_gpu_cmd_res_flush res 0x2000, w 640, h 480, x 0, y 0
[0.361] [INF] [gpu.rs@973] Rendered green frame and displayed successfully!
```

Die Ausgaben zeigen, dass (i) der 3D-Kontext erstellt wird, (ii) die 3D-Resource angelegt und an den Kontext gebunden wird, (iii) der Command-Stream mit `CMD_SUBMIT_3D` übertragen wird und (iv) der Host eine Fence-Antwort liefert. Letzteres ist ein Indikator dafür, dass die Submission verarbeitet wurde und auf einer Synchronisations-Timeline eindeutig referenzierbar ist.

Zusätzlich lässt sich die korrekte Kodierung des Command-Streams direkt über die Debug-Ausgabe der Funktion `dump_cs` nachvollziehen. Dabei werden die im Command-Stream enthaltenen 32-Bit Headerwörter dekodiert und als Tripel `(cmd, obj, len)` ausgegeben. Die Dekodierung folgt der in Abschnitt 4.2 beschriebenen Bitaufteilung, wobei `cmd` die unteren 8 Bit, `obj` die nächsten 8 Bit und `len` die oberen 16 Bit (Anzahl der folgenden 32-Bit Payload-Wörter) repräsentiert:

gpu.rs	Rust
<pre> 1 fn decode_hdr(h: u32) -> (u32,u32,u32) { 2 let cmd = h & 0xff; 3 let obj = (h >> 8) & 0xff; 4 let len = h >> 16; 5 (cmd, obj, len) 6 }</pre>	

Im Log des Smoke Tests erscheinen dadurch die folgenden dekodierten Header:

- `hdr @000: cmd=1 obj=8 len=5`
- `hdr @024: cmd=5 obj=0 len=3`
- `hdr @040: cmd=4 obj=0 len=7`
- `hdr @072: cmd=7 obj=0 len=8`

Diese Werte stimmen mit der erwarteten Encodierung der verwendeten Encoder überein. So beginnt etwa `encode_create_surface` mit einem Headerwort, das über `virgl_cmd0(VIRGL_CCMD_CREATE_OBJECT, VIRGL_OBJECT_SURFACE, 5)` erzeugt wird. Die Wertezuordnung erfolgt direkt: `VIRGL_CCMD_CREATE_OBJECT = 1` steht dabei für das Kommando `cmd=1`. Analog dazu entspricht `VIRGL_OBJECT_SURFACE = 8` dem Objekt `obj=8`. Der Parameter `len=5` beschreibt schließlich die nachfolgenden fünf 32-Bit Payload-Wörter des Encoders. Die Dump-Ausgabe liefert damit einen zusätzlichen, internen Konsistenznachweis dafür, dass der erzeugte Byte-Stream strukturell korrekt aufgebaut ist und vom Host in dieser Struktur verarbeitet werden kann.

Abschließend liefert das visuelle Ergebnis den entscheidenden funktionalen Nachweis für die korrekte Interpretation der Befehlskette. Wie bereits in Abschnitt 4.2 hergeleitet, definieren die im Testcode über `encode_clear_color` gesetzten normalisierten Fließkommawerte ($R = 0.094$, $G = 0.416$, $B = 0.067$, $A = 1.0$) nach der Skalierung in den 8-Bit-Farbraum die RGBA-Farbe (24, 106, 17, 255).

Der in Abbildung 5.1 dargestellte Framebuffer zeigt exakt diese dunkelgrüne Färbung. Da diese Farbinformation ausschließlich über den 3D-Befehl `VIRGL_CCMD_CLEAR` im Command-Stream übermittelt wurde, beweist die korrekte Darstellung, dass die gesamte Pipeline von der Submission im Gast über die Verarbeitung im Host bis zum Flush funktional intakt ist.



Abbildung 5.1: Screenshot des VirGL Smoke Tests
(R = 24, G = 106, B= 17, A = 255)

5.2 VirtIO-Sound (PCM-Playback)

Zur Validierung der VirtIO-Sound-Anbindung wurde die Beispielroutine ausgeführt. Der Test bettet eine PCM-Datei direkt in das Kernel-Image ein und konfiguriert anschließend den ersten verfügbaren Output-Stream des VirtIO-Sound-Geräts. Die Ausgabe erfolgt blockweise in periodengroßen Transfers, wobei mehrere Perioden gleichzeitig in der TX-Queue gehalten werden, um eine kontinuierliche Wiedergabe sicherzustellen und Unterläufe zu vermeiden.

Im Log ist zunächst erkennbar, dass die PCM-Fähigkeiten des Geräts erfolgreich abgefragt werden. Für den Output-Stream werden dabei u. a. eine breite Menge unterstützter Abtastraten bis hin zu 384 kHz sowie mehrere Sampleformate (S8/U8, S16/U16, S32/U32, Float) ausgegeben. Anschließend werden die Zielparameter für den Test festgelegt und protokolliert: `rate=44100`, `fmt=S16`, `ch=2`, `period=4096 B` und `buffer=32768 B`. Während der Initialisierung tritt einmalig die Warnung „Error getting chmap infos“ auf. Diese Warnung betrifft die Abfrage der Channel-Map (Zuordnung der Kanäle, z. B. links-/rechts). Für den vorliegenden Stereo-Test (`ch=2`) ist diese Information nicht zwingend erforderlich, da die Wiedergabe mit Standardzuordnung erfolgt und die PCM-Konfiguration sowie der Streamstart erfolgreich sind. Entscheidend ist, dass die Playback-Pipeline anschließend vollständig durchläuft und die Wiedergabe mit „PCM file playback finished“ abgeschlossen wird.

```
[0.125] [INF] [sound.rs@192] [sound device] pcm_info: features: PcmFeatures(0x0),
  rates: PcmRates(RATE_5512 | RATE_8000 | RATE_11025 | RATE_16000 | RATE_22050 |
  RATE_32000 | RATE_44100 | RATE_48000 | RATE_64000 | RATE_88200 | RATE_96000 |
  RATE_176400 | RATE_192000 | RATE_384000), formats: PcmFormats(S8 | U8 | S16 | U16
  | S32 | U32 | FLOAT), direction: OUTPUT
[0.127] [INF] [sound.rs@192] [sound device] pcm_info: features: PcmFeatures(0x0),
  rates: PcmRates(RATE_5512 | RATE_8000 | RATE_11025 | RATE_16000 | RATE_22050 |
  RATE_32000 | RATE_44100 | RATE_48000 | RATE_64000 | RATE_88200 | RATE_96000 |
  RATE_176400 | RATE_192000 | RATE_384000), formats: PcmFormats(S8 | U8 | S16 | U16
  | S32 | U32 | FLOAT), direction: INPUT
[0.128] [WRN] [sound.rs@204] [sound device] Error getting chmap infos
[0.129] [INF] [virtio_tests.rs@094] PCM config -> rate=Rate44100, fmt=S16, ch=2,
  period=4096 B, buffer=32768 B
[231.930] [INF] [virtio_tests.rs@154] PCM file playback finished
```

Zusätzlich lässt sich über die Zeitstempel der Logausgaben die Dauer des Tests abschätzen: Zwischen der Konfigurationsausgabe bei ca. 0.129 s und dem Abschluss bei 231.930 s liegen rund 231.8 s (≈ 3 min 52 s). Dies entspricht der Länge der verwendeten Audiodatei (3:52 Minuten, Taylor Swift – Blank Space (Lyrics)[16]) und zeigt damit, dass der Stream über die gesamte Laufzeit stabil bedient werden konnte. Insgesamt belegt der Test die End-to-End-Funktionalität des Datenpfads vom Gast (Virtqueue-Transfers) über den Hypervisor bis zur Host-Audiokette.

5.3 Leistung: Rectangle-Demo

Die Rectangle-Demo erzeugt eine konstante Renderlast, indem eine feste Anzahl farbiger Rechtecke (hier: 500 Rechtecke) kontinuierlich über den Bildschirm bewegt und pro Frame neu gerendert wird. Die FPS-Ermittlung erfolgt über Framezählung und Zeitmessung in festen Intervallen, analog zur in der Vorarbeit beschriebenen Messlogik.

Die Demo wurde für unterschiedliche Umgebungen und Auflösungen ausgeführt, wobei folgende Hardware-Konfiguration verwendet wurde:

- CPU: AMD Ryzen 7 3700x (8 Kerne, 4.20 GHz)
- GPU: AMD Radeon RX 7900 XT
- Host-Umgebung A: Ubuntu 25.10
- Host-Umgebung B: WSL2 (Ubuntu 24.04.1 LTS)
- Drei Auflösungen: 640x480, 1920×1080, 2560x1440

Jeder Testlauf wurde über 3 Minuten ausgeführt, wodurch pro Konfiguration mehrere 30-Sekunden-Messpunkte entstehen.

5.3.1 Messmethodik und Korrektur der Zeitbasis

Die Demo berechnet FPS als Quotient aus gerenderten Frames und verstrichener Zeit. In der Referenzimplementierung wird hierfür `sys_get_system_time()` verwendet, und es wird in regelmäßigen Intervallen (alle 30 Sekunden) geloggt.

In der hier betrachteten Implementierung werden bestimmte GPU-Zugriffe in kurzen kritischen Abschnitten ausgeführt, die Interrupts temporär deaktivieren (zur Vermeidung von „Interrupt Storms“ durch Demo-Thread und VirtIO-Interruptpfad). Da D3OS die Systemzeit Timer-Interrupt fortschreibt, bleibt `sys_get_system_time()` während interruptfreier Abschnitte stehen. Dadurch kann es zu einer Abweichung zwischen „Gastzeit“ und tatsächlich verstrichener Hostzeit kommen, was eine direkte FPS-Auswertung auf Basis der Gastzeit verfälschen würde.

Um die reale Bildrate zu bestimmen, wird daher pro 30-Sekunden-Messintervall zusätzlich die tatsächlich auf dem Host verstrichene Zeit t_{host} erfasst. Aus dem im Gast geloggtten FPS-Wert $\text{FPS}_{\text{guest}}$ und der Gastintervalllänge $t_{\text{guest}} = 30\text{ s}$ ergibt sich zunächst die Frameanzahl:

$$N_{\text{frames}} = \text{FPS}_{\text{guest}} \cdot t_{\text{guest}}$$

Die korrigierte, reale Bildrate berechnet sich anschließend als:

$$\text{FPS}_{\text{real}} = \frac{N_{\text{frames}}}{t_{\text{host}}}$$

Beispielrechnung: Bei $\text{FPS}_{\text{guest}} = 998\text{ fps}$ über $t_{\text{guest}} = 30\text{ s}$ entstehen $N_{\text{frames}} = 998\text{ fps} \cdot 30\text{ s} = 29940\text{ Frames}$. Wenn dafür auf dem Host $t_{\text{host}} = 39\text{ s}$ vergehen, ergibt sich $\text{FPS}_{\text{real}} = \frac{29940}{39\text{ s}} \approx 768\text{ fps}$.

Diese Korrektur wird für alle Messpunkte angewandt, bevor Mittelwerte pro Auflösung/Umgebung gebildet werden.

5.3.2 Ergebnisse

Die Messergebnisse werden pro Umgebung und Auflösung tabellarisch dargestellt. Die Tabellen zeigen dabei auch die jeweilige gemessene Zeitabweichung (t_{drift}), welche sich aus der real verstrichen Host-Zeit t_{host} und konstanter Messdauer t_{guest} ergibt. Diese Zeitabweichung beschreibt also effektiv die Dauer, in der die CPU durch synchrone GPU-Operation blockiert war. Die korrigierten FPS-Werte ergeben sich aus der zuvor gezeigten Rechnung.

Umgebung	Auflösung	t_{host}	t_{drift}	FPS _{guest}	FPS _{real}
Ubuntu25	640x480	30 s	0 s	5451	5451
Ubuntu25	1920x1080	32 s	2 s	997	935
Ubuntu25	2560x1440	39 s	9 s	998	768

Tabelle 5.1: Durchschnittliche FPS mit Ubuntu25 Host

Die Ergebnisse unter der nativen Linux-Host-Umgebung (Tabelle 5.1) demonstrieren eine hohe Effizienz bei der Verarbeitung der Grafikbefehle. Bei der Basisauflösung von 640x480 ist die Zeitabweichung nicht messbar (0 s), was auf einen vernachlässigbaren Overhead der Grafikpipeline hindeutet und extrem hohe Bildraten ermöglicht. Mit steigender Auflösung und dem damit verbundenen höheren Datendurchsatz (Framebuffer-Größe) wächst die Zeitabweichung moderat an. Bei WQHD (2560x1440) beträgt der „Blindflug“ des Gast-systems 9 s, was einer Blockierungszeit von ca. 23 % entspricht.

Im Kontrast dazu zeigt die Ausführung unter WSL2 (Tabelle 5.2) ein signifikant abweichendes Verhalten, das auf die strukturellen Unterschiede der Virtualisierungsarchitektur schließen lässt.

Umgebung	Auflösung	t_{host}	t_{drift}	FPS _{guest}	FPS _{real}
WSL2	640x480	33 s	3 s	1020	956
WSL2	1920x1080	46 s	16 s	995	649
WSL2	2560x1440	61 s	31 s	997	490

Tabelle 5.2: Durchschnittliche FPS mit WSL2 Host

5.4 Diskussion und Ausblick

Die Gegenüberstellung der Messergebnisse offenbart, dass die Performance der VirtIO-Integration stark von der zugrunde liegenden Hypervisor-Architektur abhängt. Während die Ubuntu-Umgebung (KVM-basiert) selbst bei hohen Auflösungen eine moderate Latenz aufweist, bricht die Leistung unter WSL2 (Hyper-V-basiert) bei steigender Pixelanzahl deutlich stärker ein.

Für zukünftige Iterationen des Treibers oder präzisere Profiling-Aufgaben wäre eine Umstellung der Zeitmessung auf CPU-Register zu empfehlen. Durch die Nutzung des Time Stamp Counter (TSC) mittels der Instruktion `rdtsc` (Read Time-Stamp Counter) könnte die Zeitmessung vollständig entkoppelt von Interrupts erfolgen. Der TSC zählt die CPU-Taktzyklen unabhängig vom Interrupt-Flag. Dies würde zwar nicht die physische Performance (FPS) erhöhen, jedoch die Notwendigkeit einer externen Stoppuhr zur Ermittlung von FPS_{real} eliminieren und dem Gastsystem ermöglichen, seine eigene „Verzögerung“ präzise und autonom zu quantifizieren.

Alternativ zu einer Änderung der Messmethode könnte auch die Interrupt-Steuerung der Architektur verfeinert werden. Anstatt Interrupts während kritischer Abschnitte global zu deaktivieren (was den System-Timer anhält), wäre eine selektive Maskierung des GPU-Interrupts zielführender. Durch das gezielte Deaktivieren der spezifischen Interrupt-Linie im Interrupt Controller (z. B. I/O-APIC) würden „Interrupt Storms“ effektiv verhindert, während der Timer-Interrupt weiterhin verarbeitet werden könnte. Dies würde die Integrität der Systemzeit (`sys_get_system_time()`) auch während lastintensiver Renderphasen wahren.

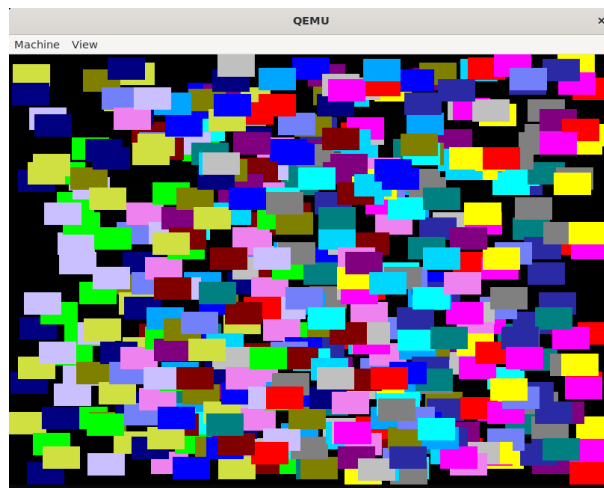


Abbildung 5.2: Screenshot der Rectangle Demo
500 Rechtecke

Kapitel 6

Fazit

Ziel dieser Bachelorarbeit war es, eine VirtIO-Treiberbasis in D3OS zu integrieren, indem die Rust-Crate `virtio-drivers` so angebunden wird, dass VirtIO-Geräte künftig über ein einheitliches und wiederverwendbares Treibermodell nutzbar sind. Dieses Ziel wurde erreicht: Die betriebssystemspezifischen Aufgaben wie Speicherbereitstellung für DMA, Adressübersetzung und die Anbindung an PCI wurden so gekapselt, dass die generische VirtIO-Logik sauber darauf aufsetzen kann. Dadurch entsteht eine tragfähige Grundlage, um weitere VirtIO-Geräte mit deutlich geringerem Implementierungsaufwand zu integrieren, statt pro Gerät eine isolierte Treiberlösung zu entwickeln.

Auf dieser Basis wurden zwei konkrete Funktionsbereiche umgesetzt und evaluiert. Für die VirtIO-GPU wurde nicht nur die grundlegende Grafikfunktionalität in D3OS verfügbar gemacht, sondern der Treiber außerdem um Mechanismen erweitert, die das Senden von VirGL/3D-Command-Streams ermöglichen. Der durchgeführte VirGL-Smoke-Test liefert dabei den End-to-End-Nachweis, dass der Befehlsfluss vom Gast bis zur Host-Verarbeitung funktioniert und Ergebnisse korrekt dargestellt werden, während gleichzeitig auch dynamische Größenänderungen (Resize) implementiert und in Demo-Programmen erprobt wurden. Die quantitative Evaluation der Grafikleistung offenbarte jedoch eine starke Abhängigkeit von der verwendeten Virtualisierungsplattform: Während ein nativer Linux-KVM-Host eine vernachlässigbare Latenz und hohe Bildraten ermöglichte, führte die Ausführung unter WSL2 (Hyper-V) bei steigenden Auflösungen zu signifikanten Leistungseinbußen. Für VirtIO-Sound wurde eine funktionierende PCM-Playback-Pipeline realisiert und über eine Beispielwiedergabe validiert, sodass Audioausgabe in der VM praktisch nutzbar ist.

Trotz der erfolgreichen Integration unterliegt die Arbeit gewissen Limitationen, insbesondere im Bereich der Leistungsmessung. Die notwendige Deaktivierung von Interrupts zur Synchronisation in kritischen Treiberabschnitten beeinträchtigte den System-Timer, was eine nachträgliche Korrektur der Zeitbasis bei Performance-Tests erforderte. Für zukünftige Weiterentwicklungen empfiehlt sich daher die Umstellung der Zeitmessung auf den CPU-internen Time Stamp Counter (TSC) sowie eine verfeinerte, selektive Interrupt-Maskierung, oder die Nutzung von Polling. Zusammenfassend lässt sich jedoch festhalten, dass die Integration von VirtIO-Treibern die Portabilität und Testbarkeit von D3OS entscheidend verbessert und eine solide Basis für zukünftige Projekte darstellt.

Literatur

- [1] Heinrich-Heine-Universität Düsseldorf. “D3OS - Research Project”, Chair of Operating Systems, besucht am 1. Feb. 2026. Adresse: <https://www.cs.hhu.de/en/research-groups/operating-systems/research/translate-to-english-hermes-1>
- [2] rCore-OS Community, *virtio-drivers: VirtIO guest drivers in Rust*, Version v0.12.0, GitHub Repository. besucht am 1. Feb. 2026. Adresse: <https://github.com/rcore-os/virtio-drivers>
- [3] IBM Cloud Education, *Was ist ein Hypervisor?*, 2024. besucht am 1. Feb. 2026. Adresse: <https://www.ibm.com/de-de/think/topics/hypervisors>
- [4] Red Hat, Inc., *Red Hat Enterprise Linux 6: Virtualization Administration Guide*, Zugriffen am: 1. Februar 2026, Raleigh, NC, USA: Red Hat, Inc., 2010. Adresse: https://www.uvm.edu/~fcs/Doc/RedHat/Red_Hat_Enterprise_Linux-6-Virtualization-en-US.pdf
- [5] E. P. Martín. “Virtqueues and virtio ring: How the data travels”, Red Hat, besucht am 25. Jan. 2026. Adresse: <https://www.redhat.com/en/blog/virtqueues-and-virtio-ring-how-data-travels>
- [6] J. Palmer. “Introduction to VirtIO, Part 2: Vhost”, Oracle, besucht am 25. Jan. 2026. Adresse: <https://blogs.oracle.com/linux/introduction-to-virtio-part-2-vhost>
- [7] E. P. Martín. “Virtio devices and drivers overview: The headjack and the phone”, Red Hat, besucht am 25. Jan. 2026. Adresse: <https://www.redhat.com/en/blog/virtio-devices-and-drivers-overview-headjack-and-phone>
- [8] OASIS Open. “Virtual I/O Device (VIRTIO) Version 1.2”, besucht am 25. Jan. 2026. Adresse: <https://docs.oasis-open.org/virtio/virtio/v1.2/csd01/virtio-v1.2-csd01.html>
- [9] E. P. Martín. “Packed virtqueue: How to reduce overhead with virtio”, Red Hat, besucht am 25. Jan. 2026. Adresse: <https://www.redhat.com/en/blog/packed-virtqueue-how-reduce-overhead-virtio>
- [10] Mesa 3D Project. “VirGL”, besucht am 25. Jan. 2026. Adresse: <https://docs.mesa3d.org/drivers/virgl.html>
- [11] The Virglrenderer Project. “vrend_decode.c”, Freedesktop.org, besucht am 25. Jan. 2026. Adresse: https://gitlab.freedesktop.org/virgl/virglrenderer/-/blob/main/src/vrend/vrend_decode.c?ref_type=heads

- [12] D. Bassey und M. V. Larsen. “Making VirtIO sing: Implementing virtio-sound in rust-vmm project (Slides)”. Virtualization and Cloud Infrastructure devroom, Brussels, FOSDEM, besucht am 26. Jan. 2026. Adresse: https://archive.fosdem.org/2024/events/attachments/fosdem-2024-1910-making-virtio-sing-implementing-virtio-sound-in-rust-vmm-project/slides/22175/implementing_virtio-sound_in_rust-vmm_project_HhqbjPY.pdf
- [13] The virtio-drivers Developers. “Trait Hal - Rust”, besucht am 25. Jan. 2026. Adresse: https://docs.rs/virtio-drivers/latest/virtio_drivers/trait.Hal.html
- [14] The virtio-drivers Developers. “virtio_drivers::transport::pci::bus::BarInfo - Rust”, besucht am 25. Jan. 2026. Adresse: https://docs.rs/virtio-drivers/latest/virtio_drivers/transport/pci/bus/enum.BarInfo.html
- [15] The Virglrenderer Project. “virgl_protocol.h”, Freedesktop.org, besucht am 25. Jan. 2026. Adresse: https://gitlab.freedesktop.org/virgl/virglrenderer/-/blob/main/src/virgl_protocol.h?ref_type=heads
- [16] T. Swift. “Blank Space (Lyrics)”, besucht am 25. Jan. 2026. Adresse: <https://www.youtube.com/watch?v=PqlwecUBrYs>

Abbildungsverzeichnis

2.1	Interaktion zwischen VirtIO-Treiber und Gerät (eigene Darstellung nach [6])	6
2.2	Der vollständige Zyklus eines Puffers in einer Split Virtqueue [5]	10
2.3	Referenzarchitektur des VirGL-Renderings im Linux-Grafikstack (Mesa) (eigene Darstellung nach [6])	14
2.4	Datenfluss der VirtIO-Sound TX-Queue (Playback) [12]	16
5.1	Screenshot des VirGL Smoke Tests (R = 24, G = 106, B= 17, A = 255) .	44
5.2	Screenshot der Rectangle Demo 500 Rechtecke	48

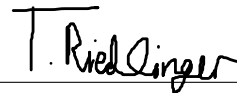
Tabellenverzeichnis

5.1	Durchschnittliche FPS mit Ubuntu25 Host	47
5.2	Durchschnittliche FPS mit WSL2 Host	47

Ehrenwörtliche Erklärung

Hiermit versichere ich, die vorliegende Bachelorarbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt zu haben. Alle Stellen, die aus den Quellen entnommen wurden, sind als solche kenntlich gemacht worden. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Düsseldorf, 02. Februar 2026

A handwritten signature in black ink, reading "T. Riedlinger", written over a horizontal line.

Tim Riedlinger